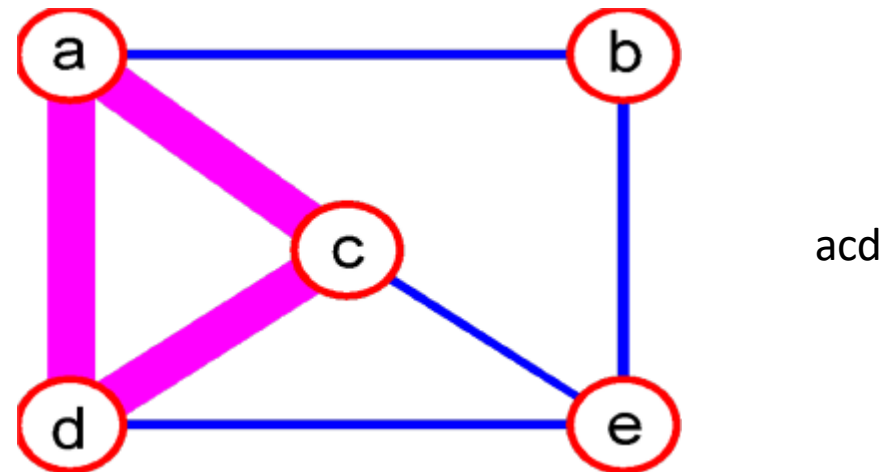
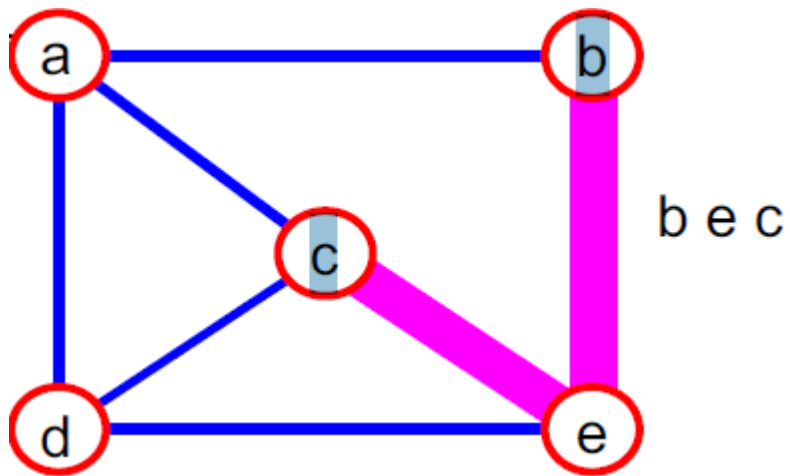


DATA STRUCTURES III



Introduction –Terminology

- Simple **path**: no repeated vertices
- **cycle**: simple path, except that the last vertex is the same as the first vertex





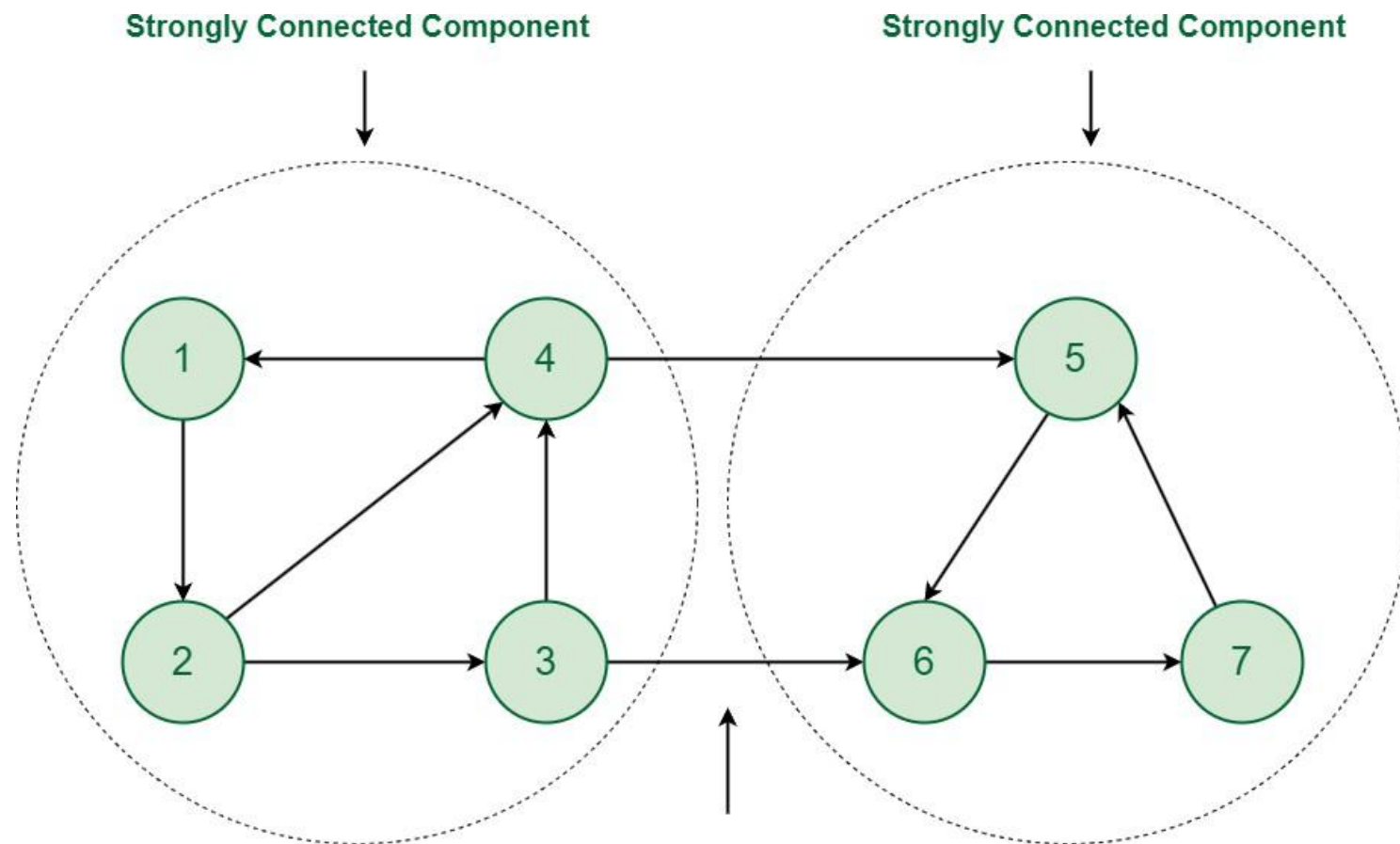
Introduction –Strongly connected components

- A graph is **connected** if given any two vertices u and v , there is a path from u to v
- A directed graph is strongly connected if for each pair of vertices there is a path between them in both directions.
- strongly connected (**every vertex is reachable from every other vertex**),
- The entire digraph is not always strongly connected, but it may be composed of **strongly connected components** (SCC), which are defined as **subsets of vertices of the graph such that each of these subsets induces a strongly connected digraph**



Strongly connected graphs

- A **strongly connected component** of a directed graph is a maximal subgraph where every pair of vertices is mutually reachable.
- This means that for any two nodes A and B in this subgraph, there is a path from A to B and a path from B to A.

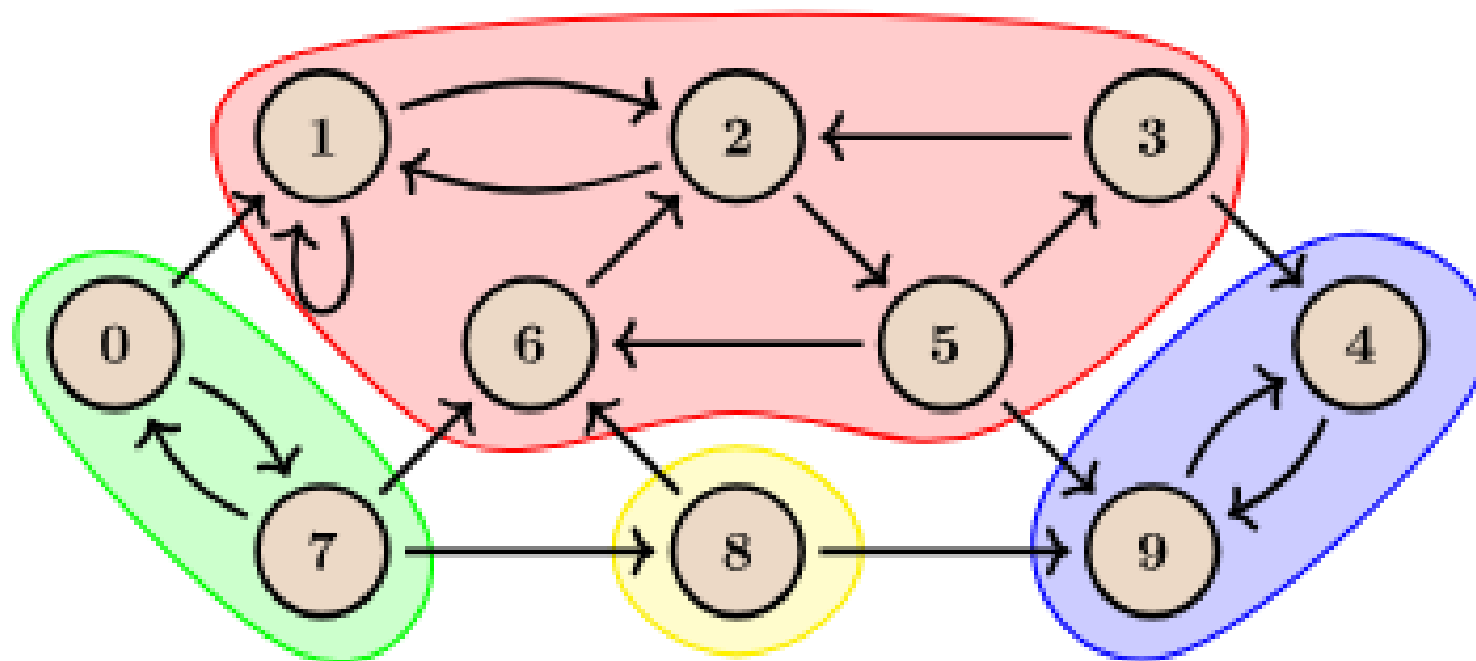


There is only a Unidirectional Edge between 2 Strongly Connected Component. Thus the two Strongly Connected Components doesn't become a single Strongly Connected Component.



Strongly connected graphs

$$\text{SCC}(G_{\text{example}}) = \{\{0, 7\}, \{1, 2, 3, 5, 6\}, \{4, 9\}, \{8\}\}$$





Strongly connected graphs

Algorithms for finding SCC

1. **Tarjan's Algorithm**
2. **Kosaraju's Algorithm**



- Breadth-first search is a **graph traversal algorithm** that starts traversing the graph from the root node and explores all the neighboring nodes.
- It starts at the root of the graph and **visits all nodes at the current level before moving on to the nodes at the next depth level.**
- It selects a single node in a graph and, after that, visits all the nodes adjacent to the selected node.
- A boolean visited array is used to mark the visited vertices.
- **BFS uses a queue data structure for traversal.**



BFS Algorithm

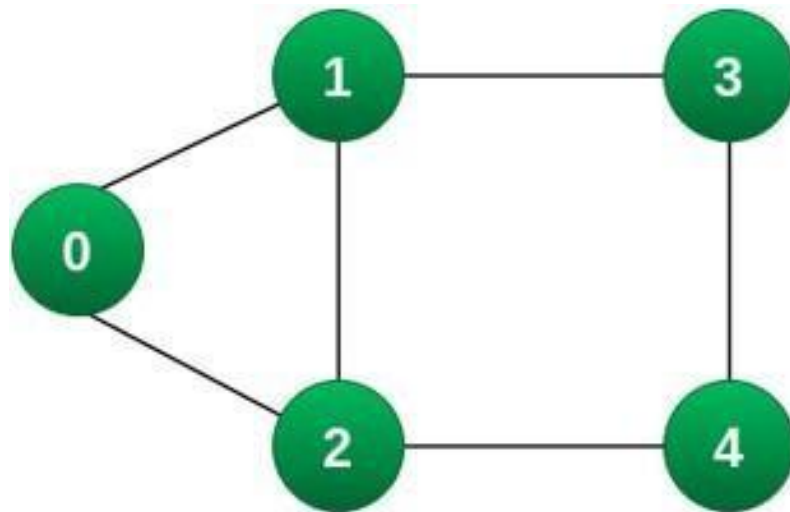
- BFS
- Initialize Q to be empty;
- Enqueue(Q,1) and mark 1;
- while Q is not empty do
 - $i := \text{Dequeue}(Q)$;
 - for each j adjacent to i do
 - if j is not marked then
 - Enqueue(Q,j) and mark j;
- end{BFS}



- Breadth-First Search (BFS)
 - – Use a queue to explore neighbors of source vertex, then neighbors of neighbors etc.
 - – All nodes at a given distance (in number of edges) are explored before we go further



- Consider graph given below. Apply BFS for traversal



Visited



Queue

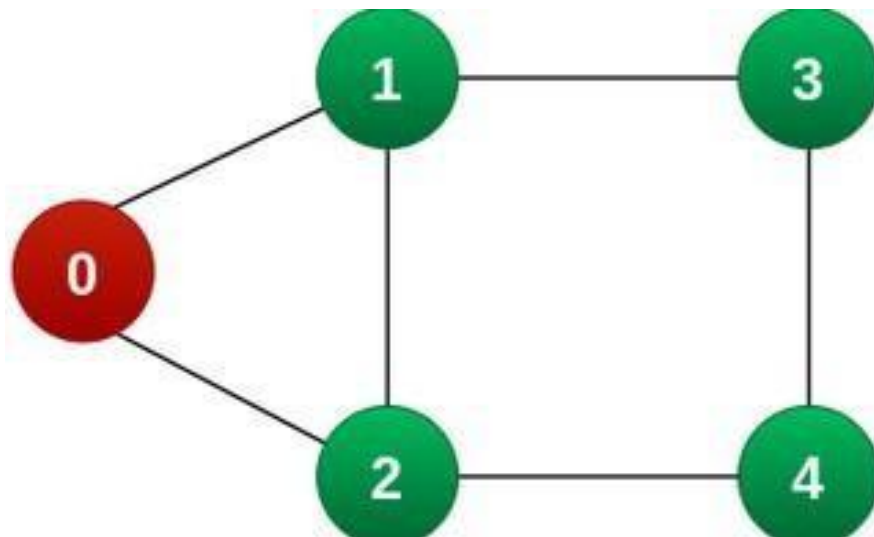


↑
FRONT



BFS

Push node 0 into queue and mark it visited.



Visited



Queue

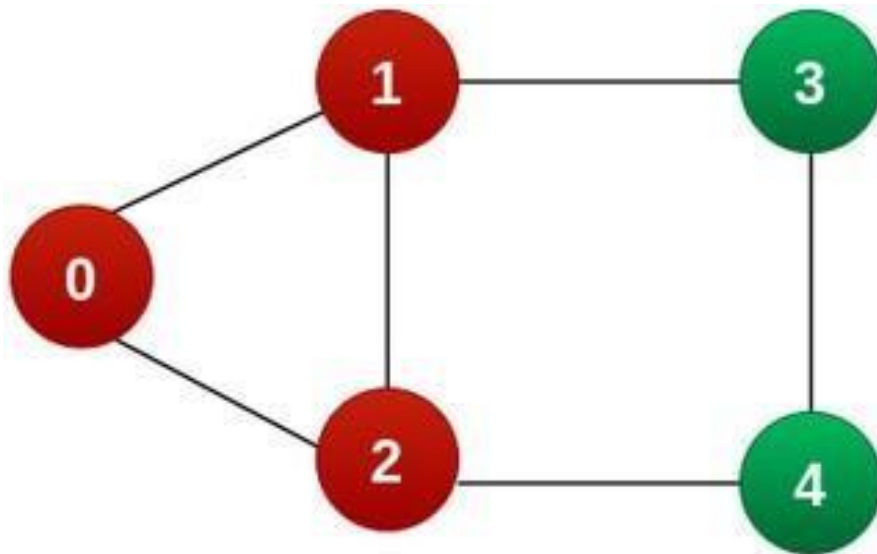


↑
FRONT



BFS

Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.



Visited

0	1	2		
---	---	---	--	--

Queue

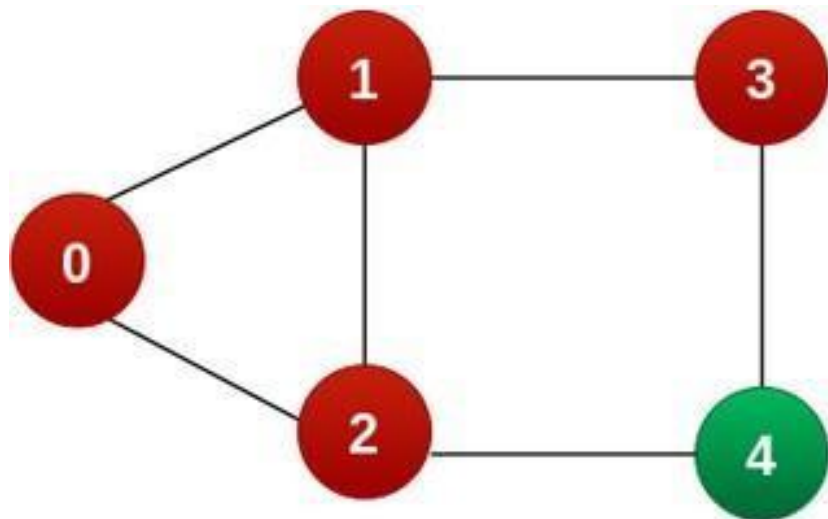
1	2			
---	---	--	--	--

↑
FRONT



BFS

- Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



Visited

0	1	2	3	
---	---	---	---	--

Queue

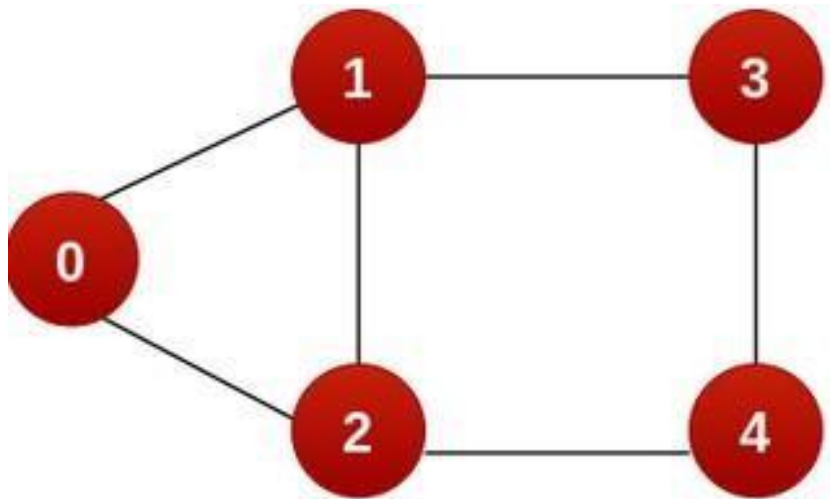
2	3			
---	---	--	--	--

↑
FRONT



BFS

- Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



Visited

0	1	2	3	4
---	---	---	---	---

Queue

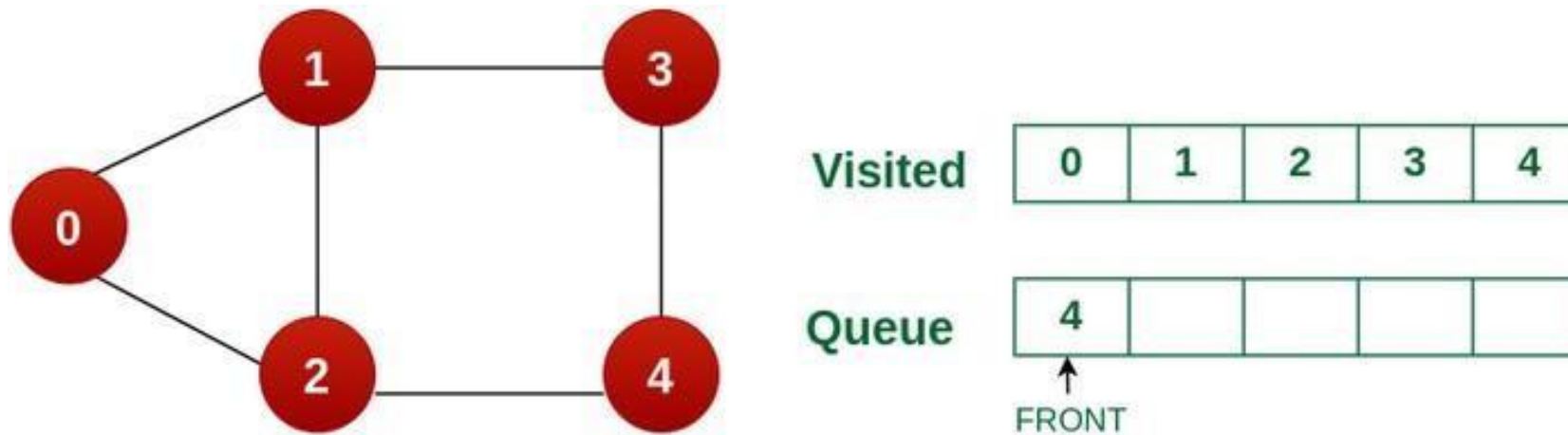
3	4			
---	---	--	--	--

↑
FRONT



BFS

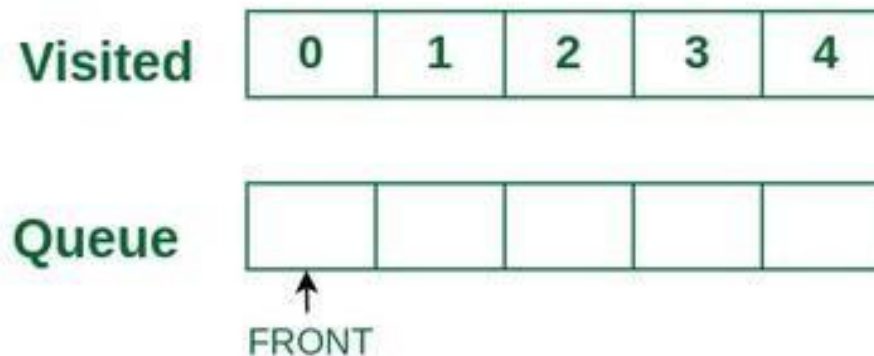
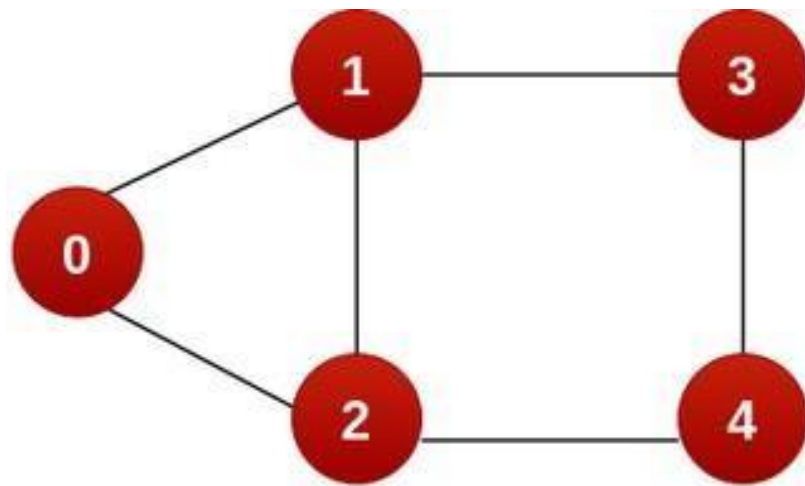
- Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.
- As we can see that every neighbours of node 3 is visited, so move to the next node that are in the front of the queue.





BFS

- Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.
- As we can see that every neighbours of node 4 are visited, so move to the next node that is in the front of the queue.



Graph is traversed

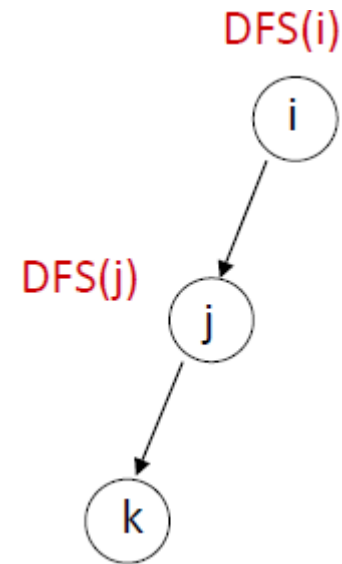


- Depth-First Search (DFS)
 - – Searches down one path as deep as possible
 - – When no nodes are available, it backtracks
 - – When backtracking, it explores side-paths that were not taken
 - – Uses a **stack (instead of a queue in BFS)**
 - – Allows an easy recursive implementation



DFS (Recursive ALGORITHM)

- Initially every vertex is unmarked– Searches down one path as deep as possible
- DFS(i: vertex)
 - mark i;
 - for each j adjacent to i do
 - if j is unmarked then DFS(j)
- end{DFS}
- Mark all vertices reachable from i





Algorithm 1: Recursive DFS

Data: G : The graph stored in an adjacency list

$root$: The starting node

Result: Prints all nodes inside the graph in the *DFS* order

$visited \leftarrow \{false\};$

$DFS(root);$

Function $DFS(u)$:

if $visited[u] = true$ **then**

return;

end

$print(u);$

$visited[u] \leftarrow true;$

for $v \in G[u].neighbors()$ **do**

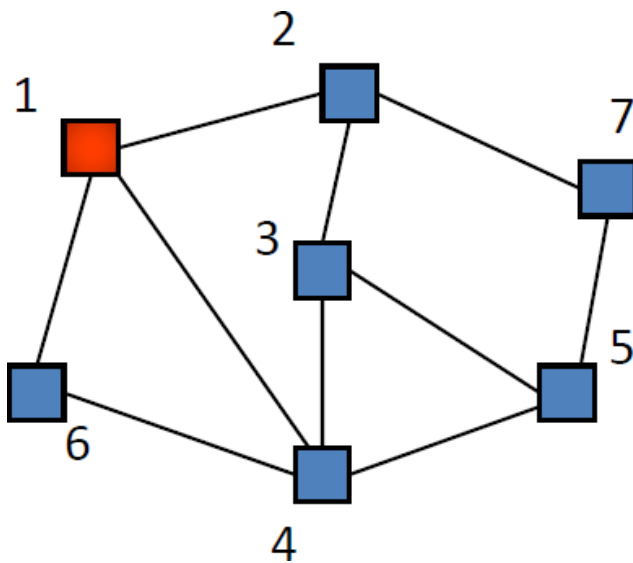
$DFS(v);$

end

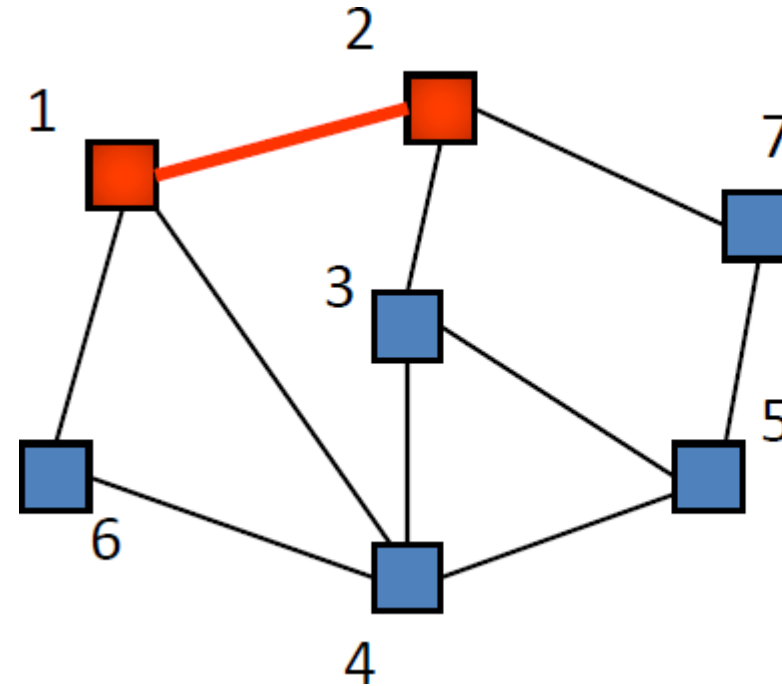
end



DFS (Recursive)



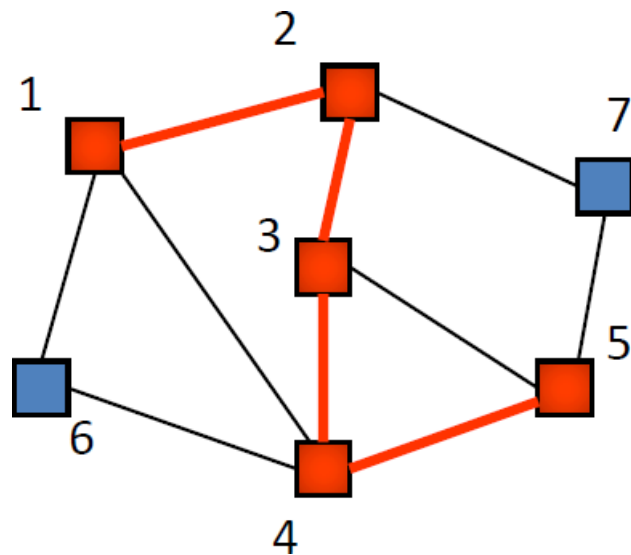
DFS(1)



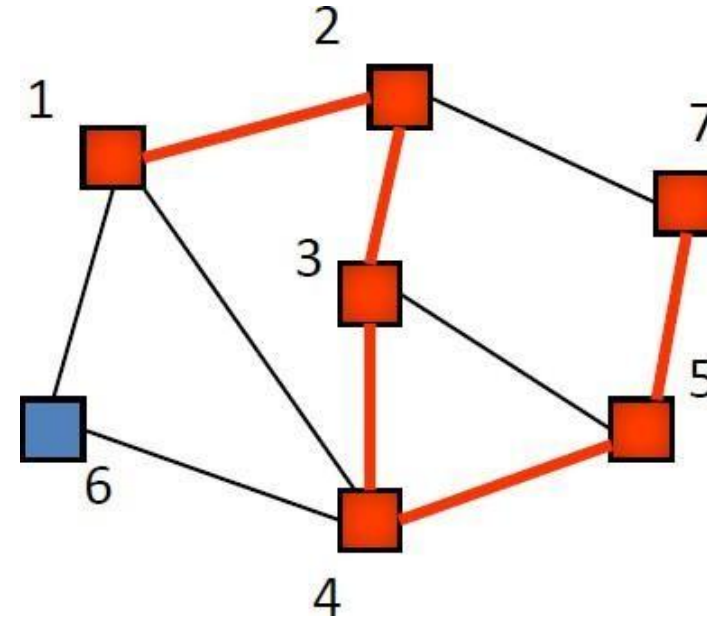
DFS(1)
DFS(2)



DFS



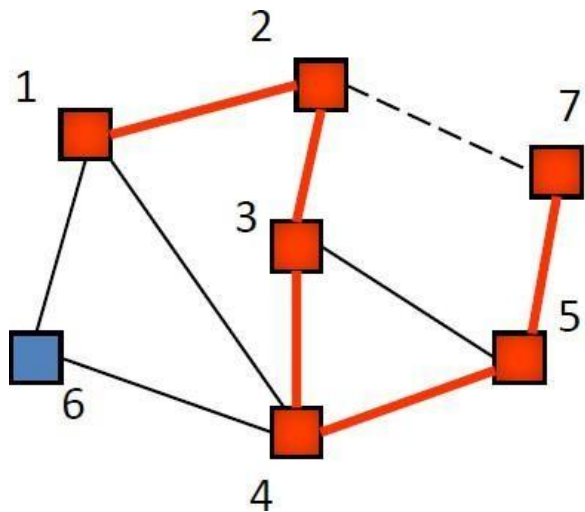
DFS(1)
DFS(2)
DFS(3)
DFS(4)
DFS(5)



DFS(1)
DFS(2)
DFS(3)
DFS(4)
DFS(5)
~~DFS(3)~~
DFS(7)

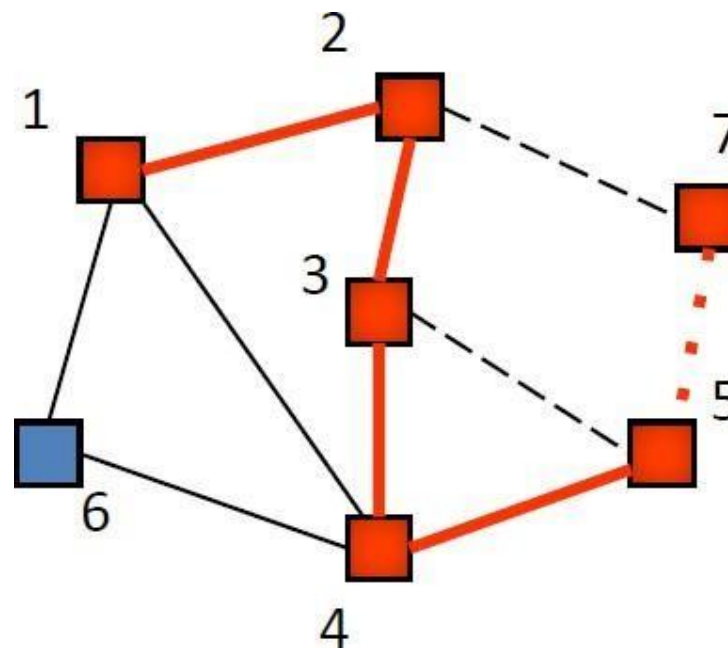


DFS



DFS(1)
DFS(2)
DFS(3)
DFS(4)
DFS(5)
DFS(7)

Now back up.

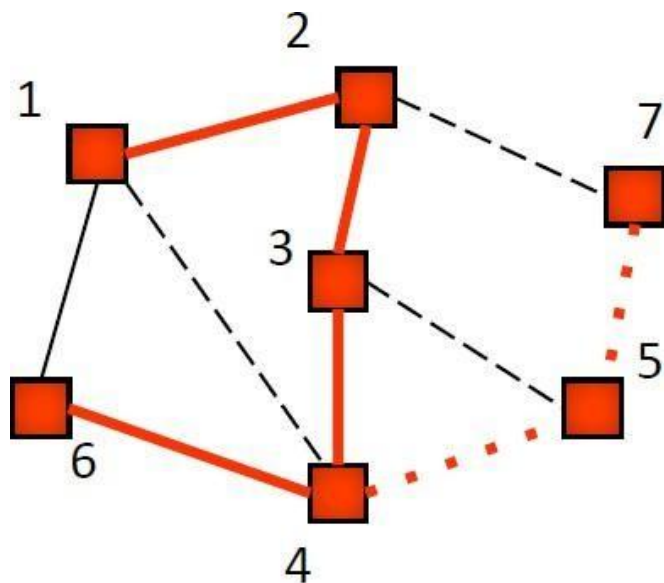


DFS(1)
DFS(2)
DFS(3)
DFS(4)
DFS(5)

Back to 5,
but it has no
more neighbors.

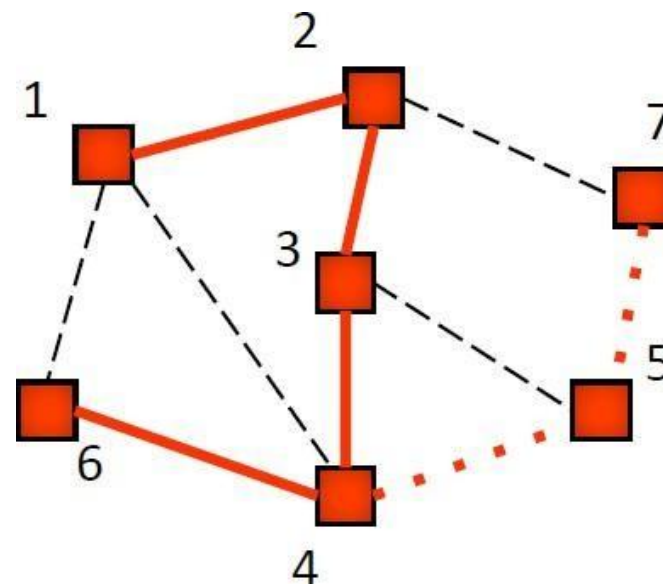


DFS



DFS(1)
DFS(2)
DFS(3)
DFS(4)
DFS(6)

Back up to 4.
From 4 we can
get to 6.

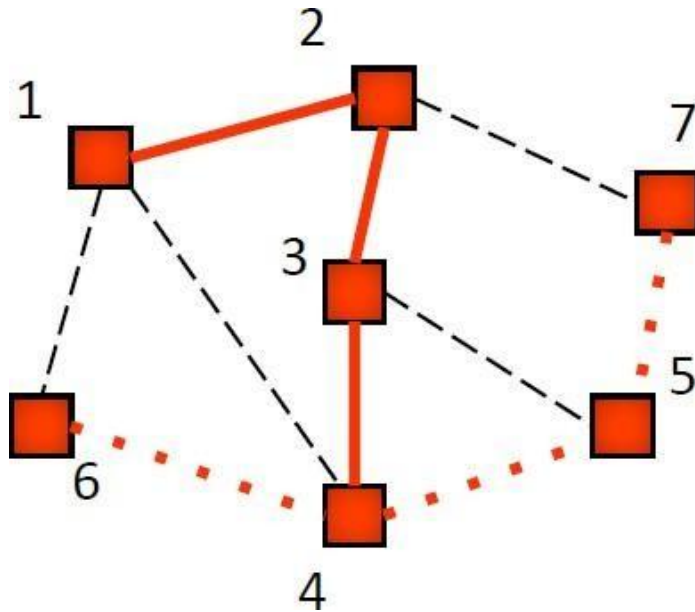


DFS(1)
DFS(2)
DFS(3)
DFS(4)
DFS(6)

From 6 there is
nowhere new
to go. Back up.

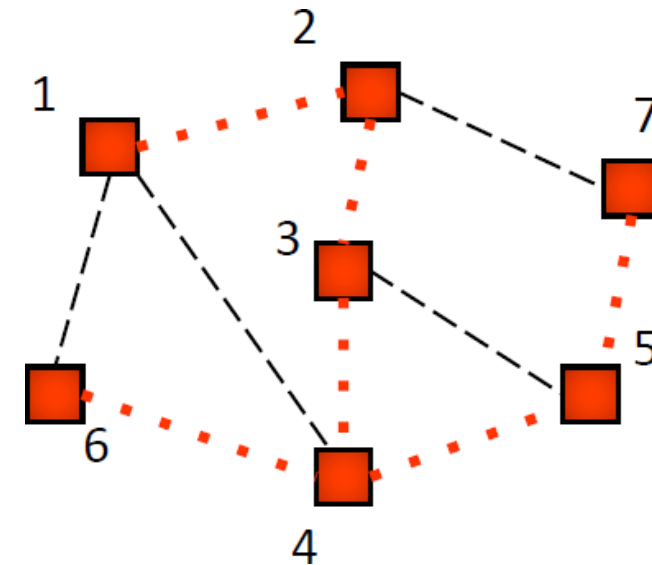


DFS



DFS(1)
DFS(2)
DFS(3)
DFS(4)

Back to 4.
Keep backing up.



DFS(1)

All the way
back to 1.

Done.

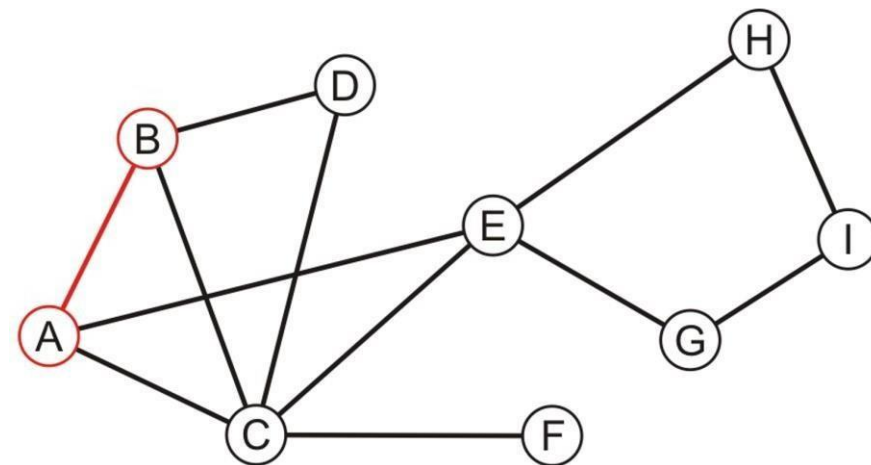
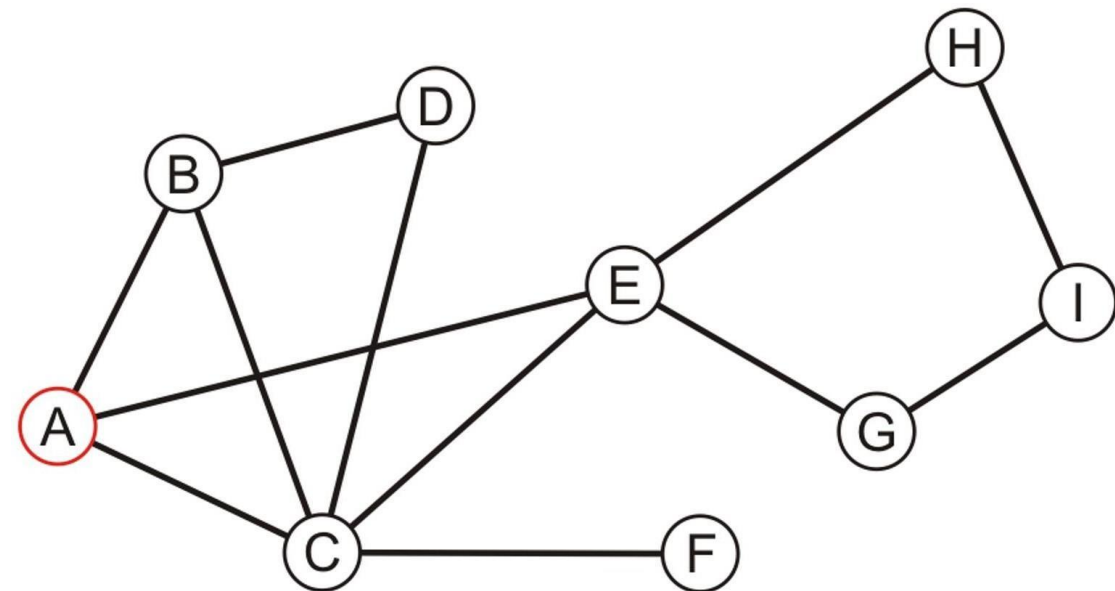
All nodes are marked so graph is connected; red links define a spanning tree

A spanning tree of a graph on n vertices is a subset of $n-1$ edges that form a tree



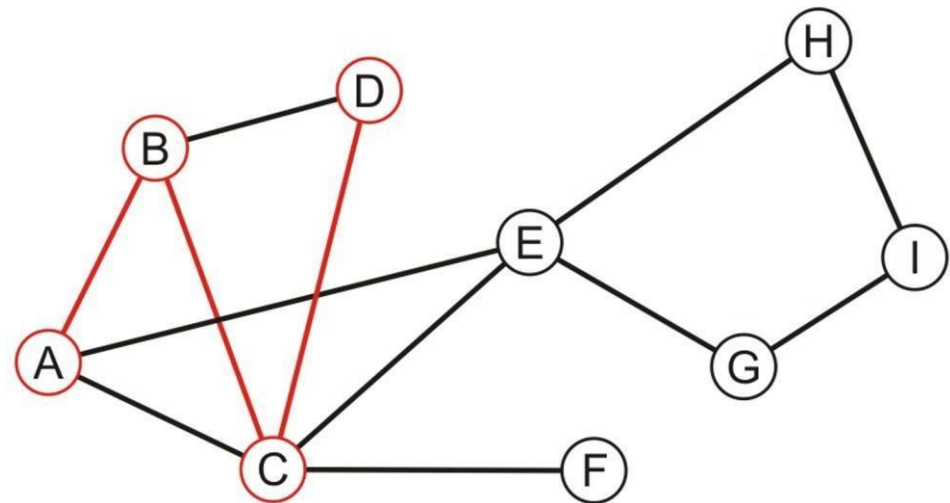
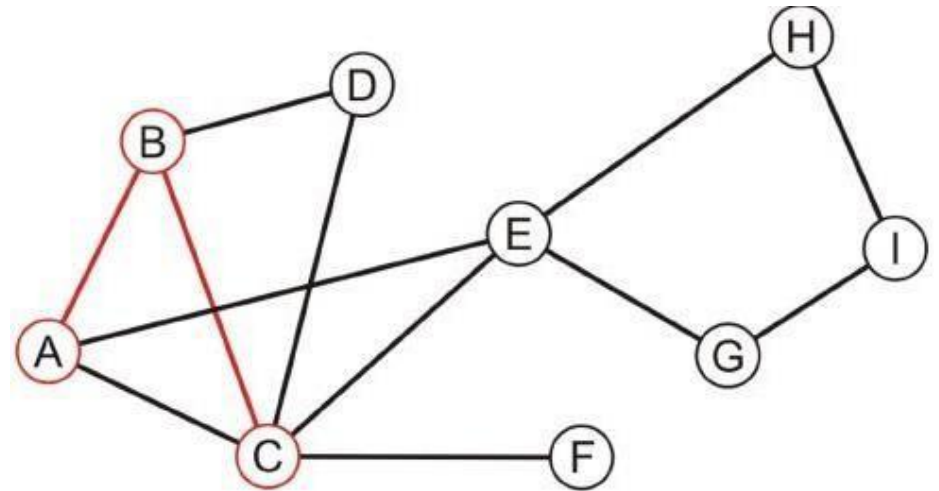
DFS

- Visit the first node A
- A has unvisited neighbors B C
- Mark A as visited :
- **VISITED :A**
- Visit node B. Mark B as visited :POP B
- **VISITED :A B**



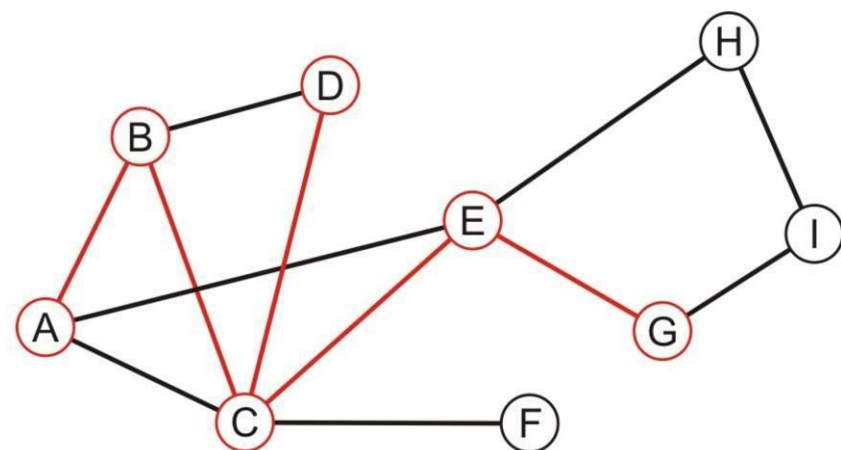
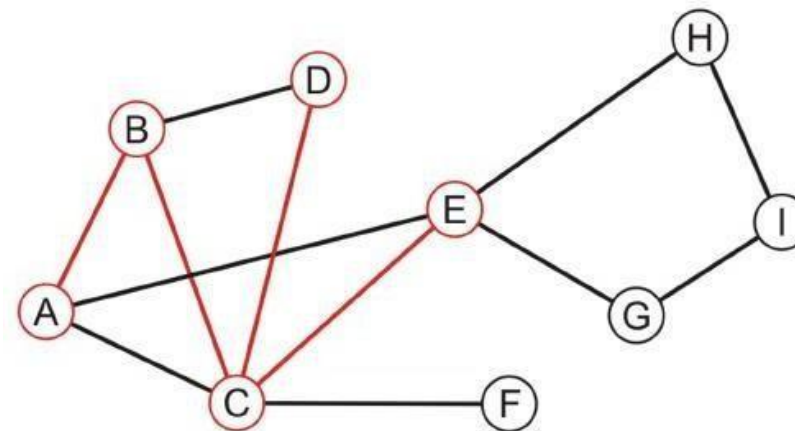


- Visit C. Mark C as visited: POP C
- **VISITED: A B C**
- Visit D. Mark D as visited: POP D
- **VISITED: A B C D**





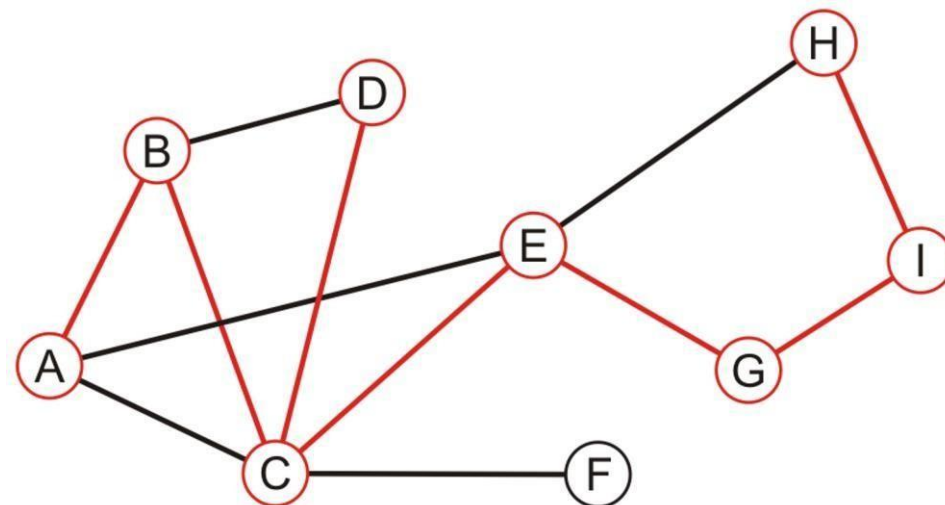
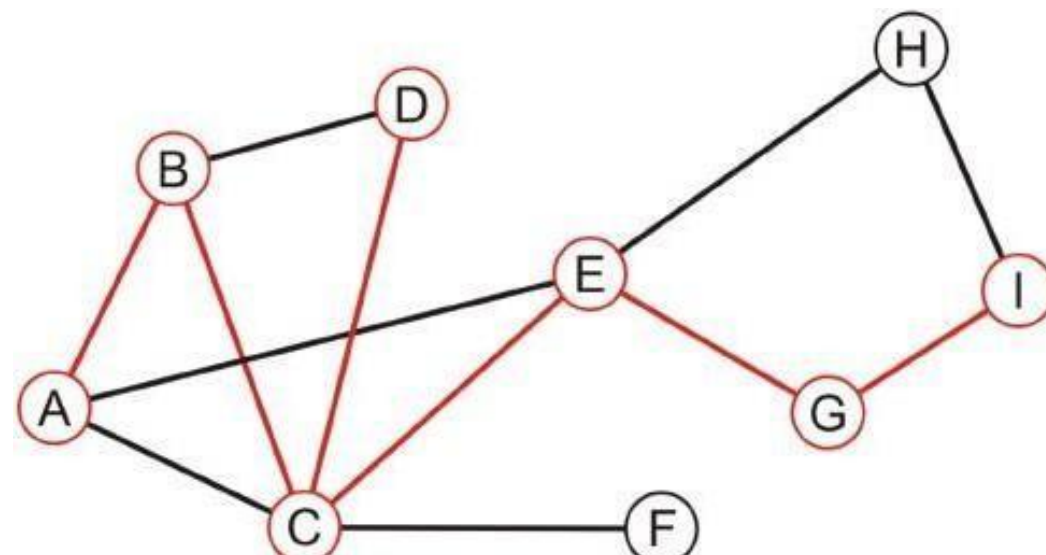
- D has no unvisited neighbors, so we return to C. C has an unvisited neighbor
- Visit E. Mark E as visited. POP E
- **VISITED: A B C D E**
- Visit G. Mark G as visited. POP G
- **VISITED: A B C D E G**





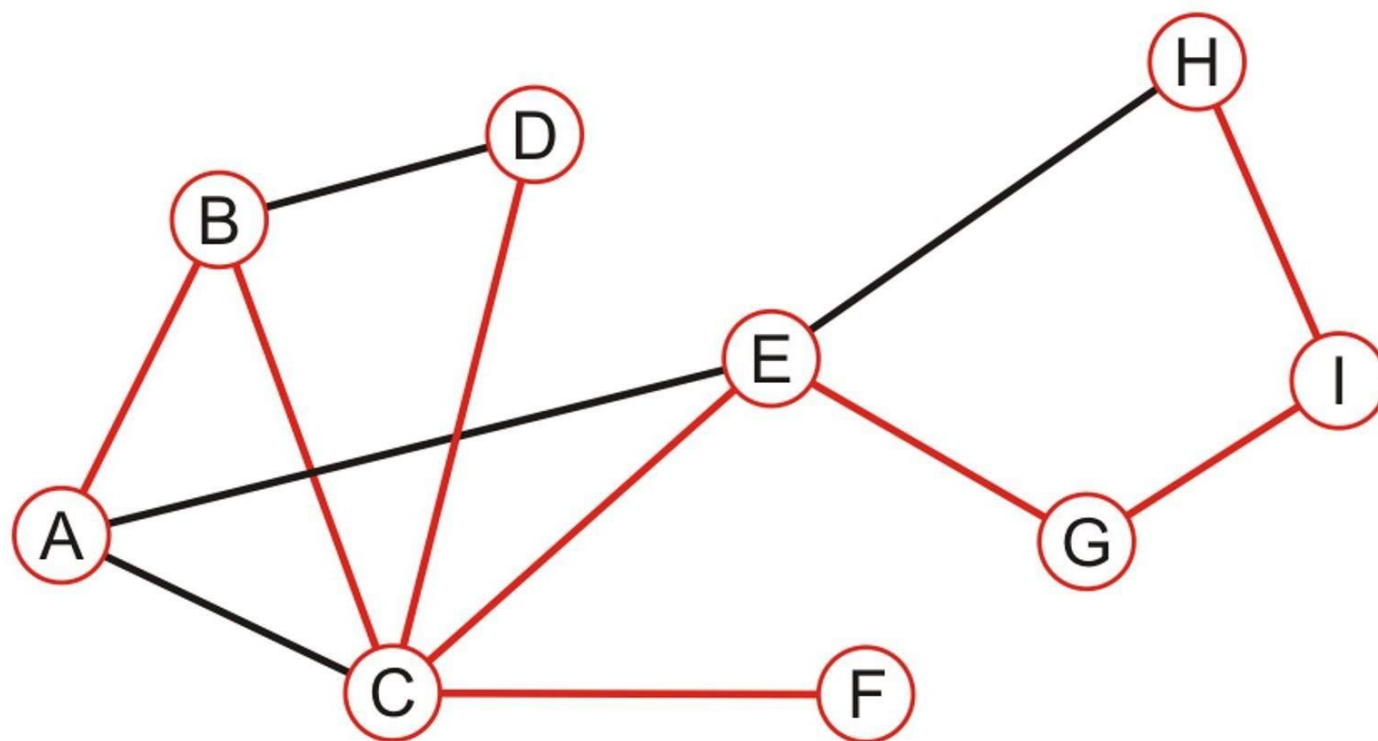
DFS

- Visit node I. Mark I as visited. POP I
- **VISITED: A B C D E G I**
- Visit node H. Mark H as visited. POP H
- **VISITED: A B C D E G I H**
- H has no unvisited neighbour, Return to I
- G has no unvisited neighbour. Return to E
- I has no unvisited neighbour, Return to G
- G has no unvisited neighbour. Return to E
- E has no unvisited neighbour.
- Return to C which has an unvisited neighbor





- Back track to C which has an unvisited neighbor
- Visit node F. Mark F as visited: POP F
- **VISITED: A B C D E G I H F**
- F has no unvisited neighbors
- STACK IS EMPTY
- **STACK:**
- Graph is traversed fully





DFS (Iterative Algorithm)

Algorithm 2: Iterative DFS

Data: G : The graph stored in an adjacency list

root: The starting node

Result: Prints all nodes inside the graph in the *DFS* order

$visited \leftarrow \{false\};$

$stack \leftarrow \{\};$

$stack.push(root);$

while $\neg stack.empty()$ **do**

$u \leftarrow stack.top();$

$stack.pop();$

if $visited[u] = true$ **then**

continue;

end

$print(u);$

$visited[u] \leftarrow true;$

for $v \in G[u]$ **do**

if $visited[v] = false$ **then**

$DFS(v);$

end

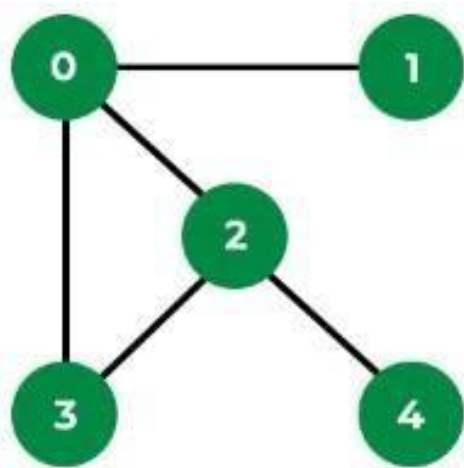
end

end



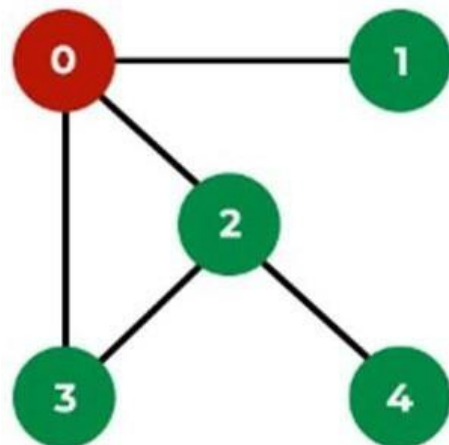
DFS (iterative)

- Perform a depth-first traversal on this graph



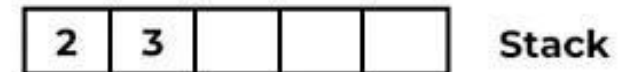
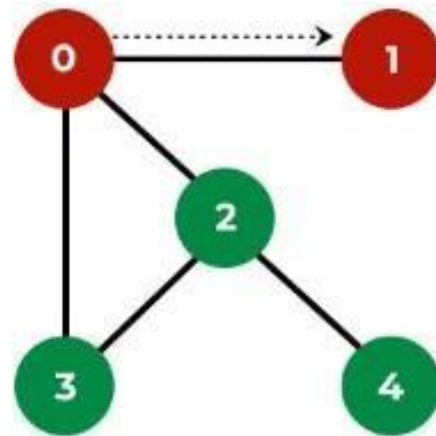


Visit 0 and put its adjacent nodes which are not visited yet into the stack.



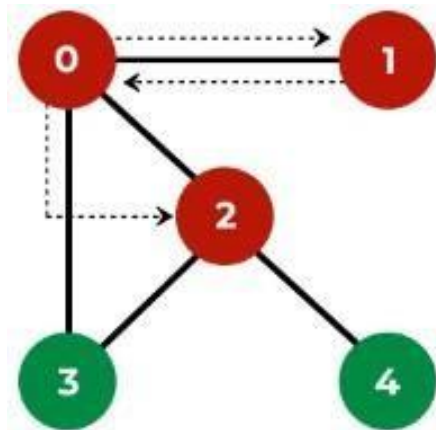


Now, Node 1 at the top of the stack, so visit node 1 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.





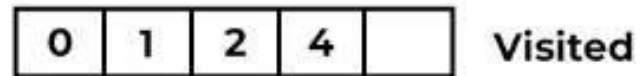
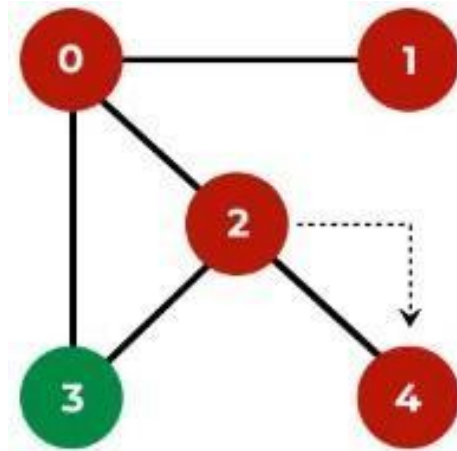
Now, Node 2 at the top of the stack, so visit node 2 and pop it from the stack and put all of its adjacent nodes which are not visited (i.e, 3, 4) in the stack.





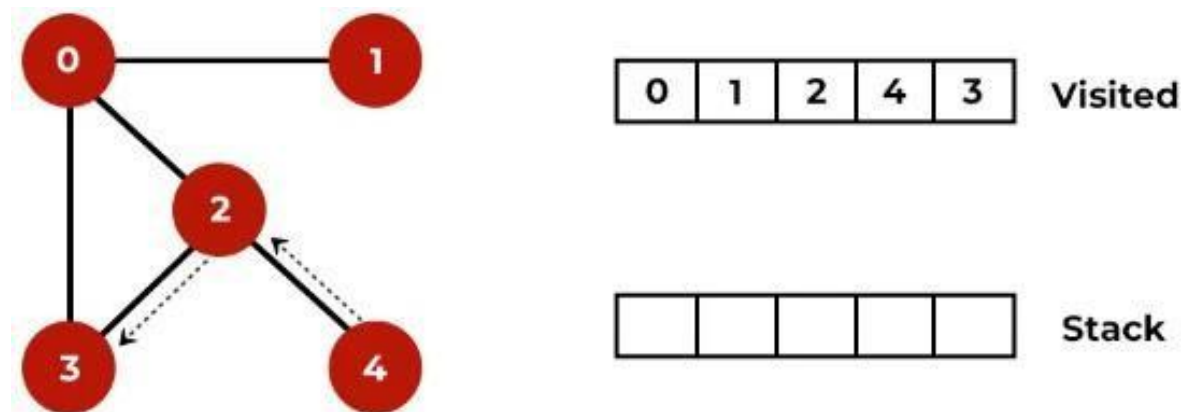
DFS

Now, Node 4 at the top of the stack, so visit node 4 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.





- Now, Node 3 at the top of the stack, so visit node 3 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



Stack becomes empty, which means we have visited all the nodes and our DFS traversal ends.



PERFORMANCE OF BFS AND DFS

- Performance
- V vertices and E edges
- Time complexity $O(V + E)$
- Space complexity $O(V)$ in worst case
-
- Big O NOTATION-An upper bound on a function which ensures that the function never grows faster than the upper bound. So, it gives the least upper bound on a function so that the function never grows faster than this upper bound.
- **Time Complexity**
- The time complexity of an algorithm describes **how the algorithm's runtime grows** as the input size increases.
- $O(n)$ means the algorithm's runtime grows linearly with the input size
- **Space complexity**
- The space complexity of an algorithm describes **how much additional space or memory** the algorithm requires to solve a problem as the input size increases.

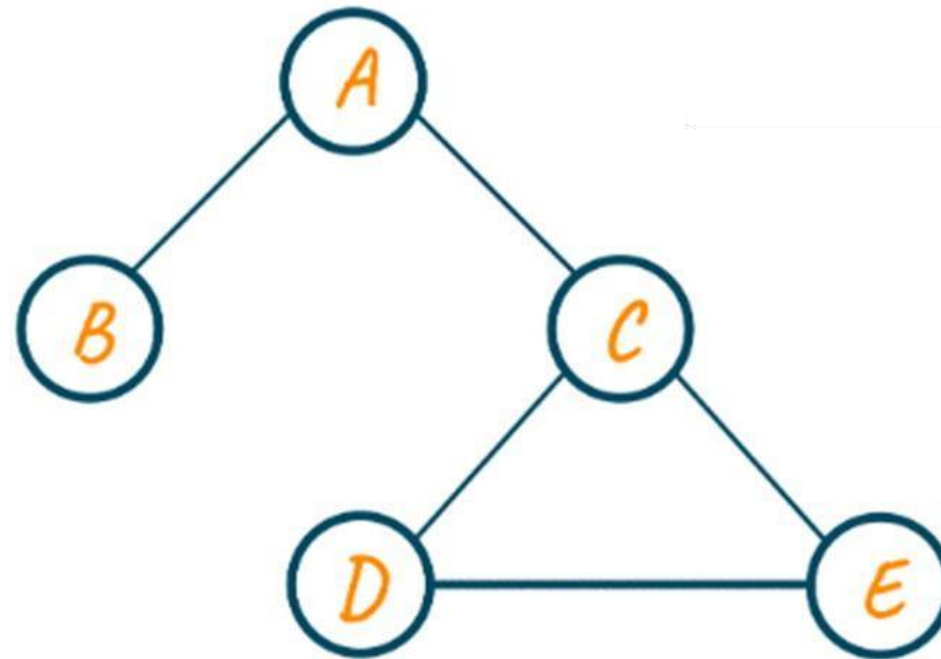


- Let G be a graph with n vertices and m edges. A DFS traversal of G takes time $O(n + m)$. Also, there exist $O(n + m)$ time algorithms based on DFS for the following problems:
 - – Testing whether G is connected.
 - – Computing a spanning tree of G
 - – Computing the connected components of G



DETECT CYCLE IN UNDIRECTED GRAPH USING DFS

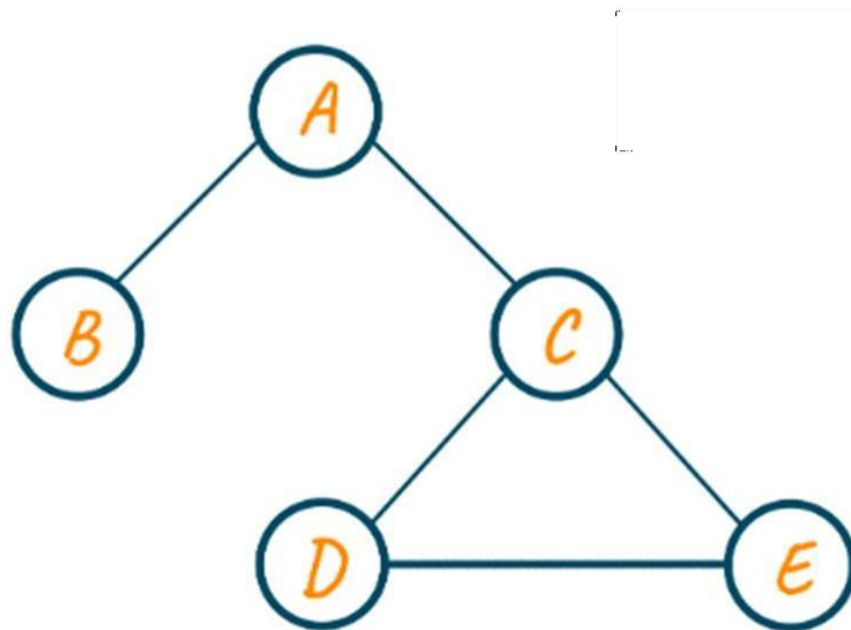
- The path which starts from any given vertex and ends at the same vertex is called the cycle of the graph.





DETECT CYCLE IN UNDIRECTED GRAPH USING DFS

Methodology to detect cycle using DFS Algorithm



- To detect if there is any cycle in the undirected graph or not, we will use the DFS traversal for the given graph.
- For **every visited vertex v** , when we have found any adjacent vertex u , such that u is already visited, and u is not the parent of vertex v . Then one cycle is detected.

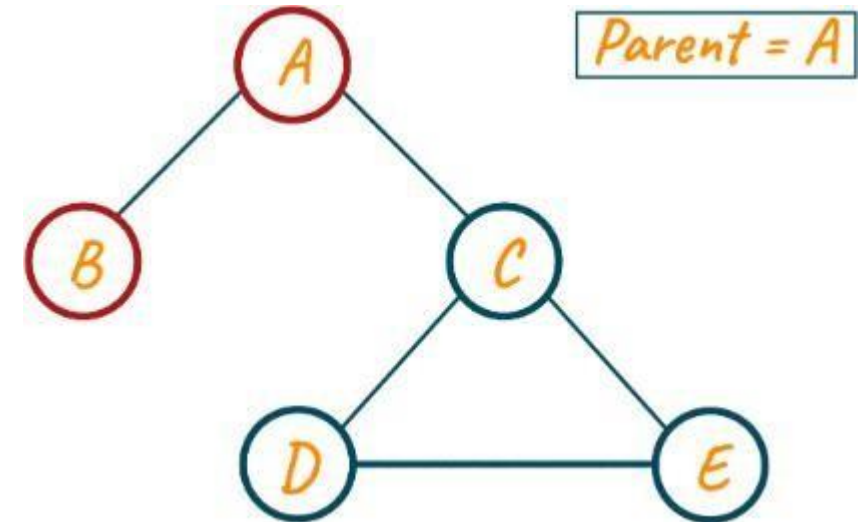
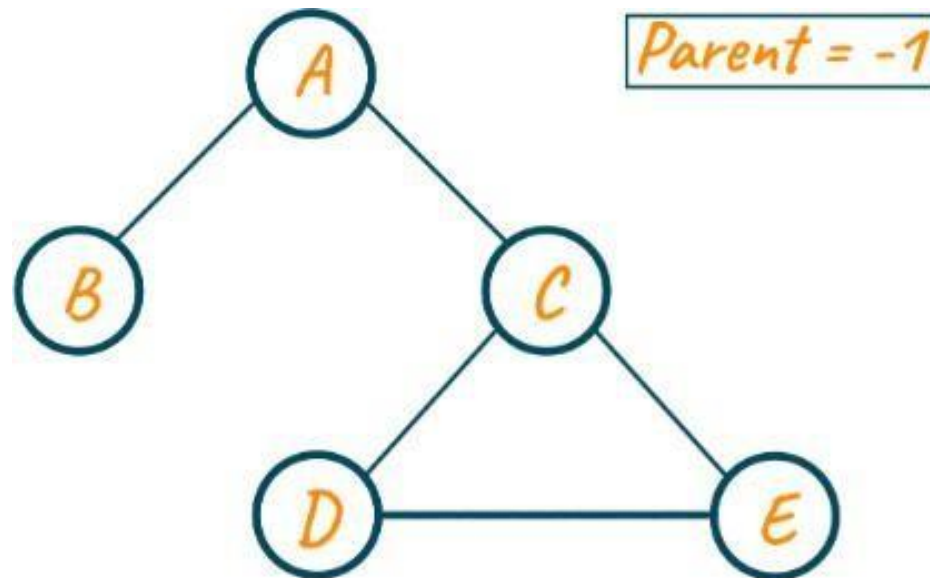


DETECT CYCLE IN UNDIRECTED GRAPH USING DFS

Algorithm

Visit node A. Run DFS.

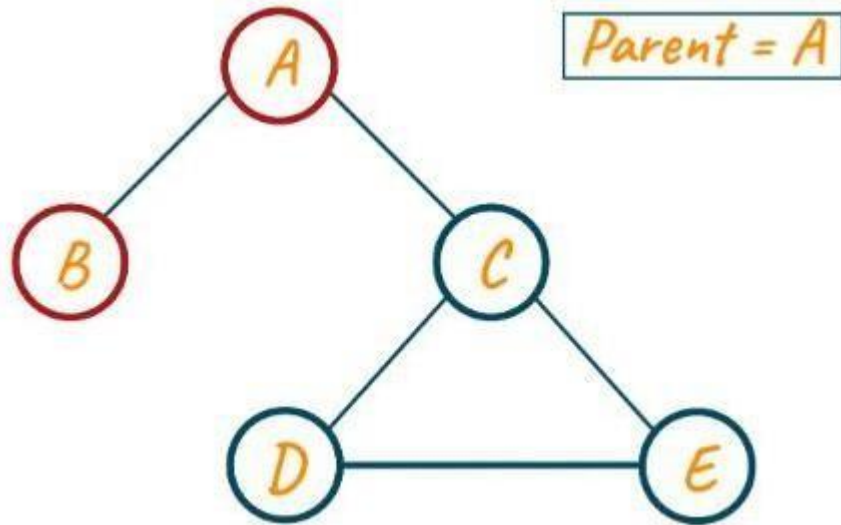
Visit Node B



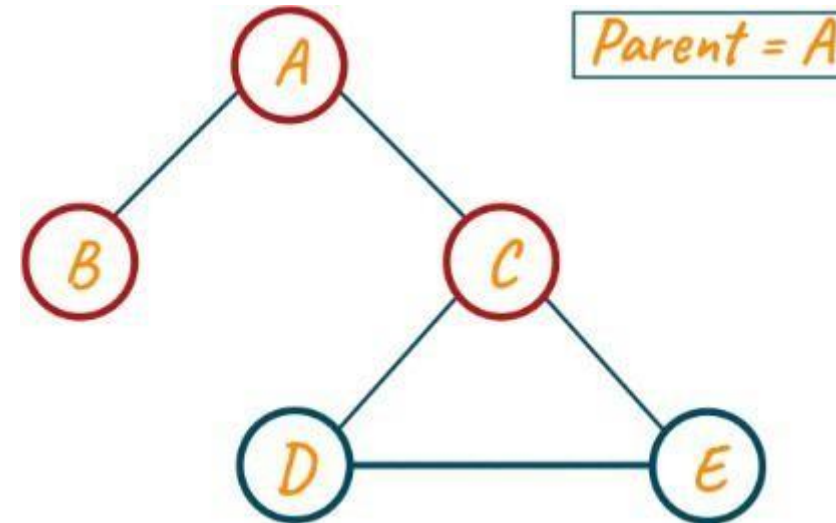


DETECT CYCLE IN UNDIRECTED GRAPH USING DFS

Run DFS from Node B.
Only neighbour is A -Parent of B
Back track to A



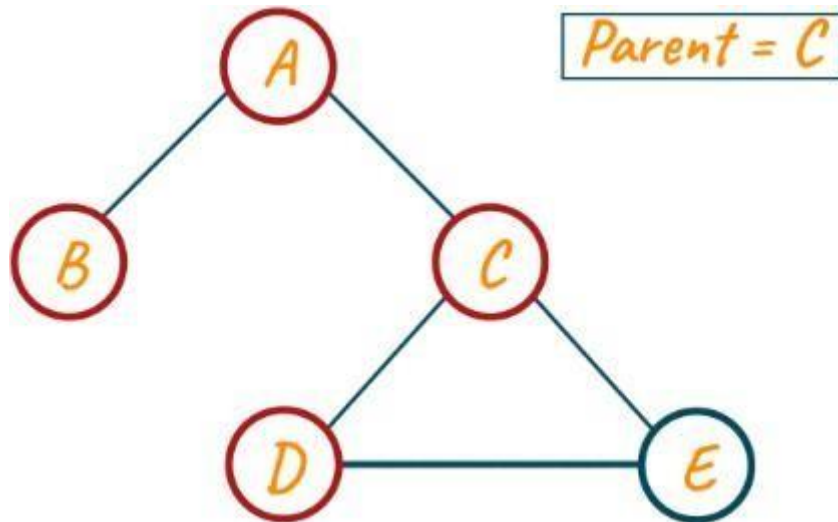
From Node A run DFS. Unvisited neighbour is C
So visit node C



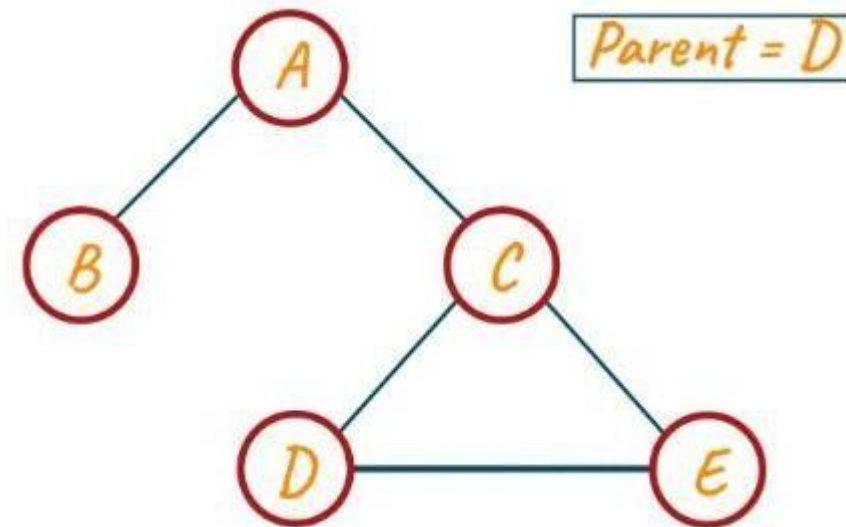


DETECT CYCLE IN UNDIRECTED GRAPH USING DFS

Run DFS from Node C.
Visit D



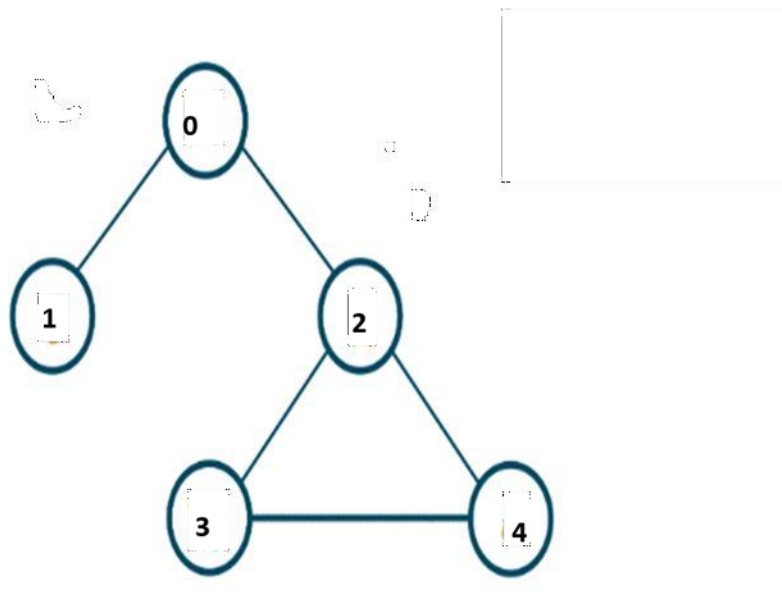
Node D has two neighbours.
C is already visited.
Visit E





DETECT CYCLE IN UNDIRECTED GRAPH USING DFS

DETECT IF CYCLE EXISTS IN THE UNDIRECTED GRAPH.
IF IT EXISTS, DISPLAY YES AND DISPLAY THE VERTICES IN THE
CYCLE
IF CYCLE DOESN'T EXISTS, DISPLAY NO





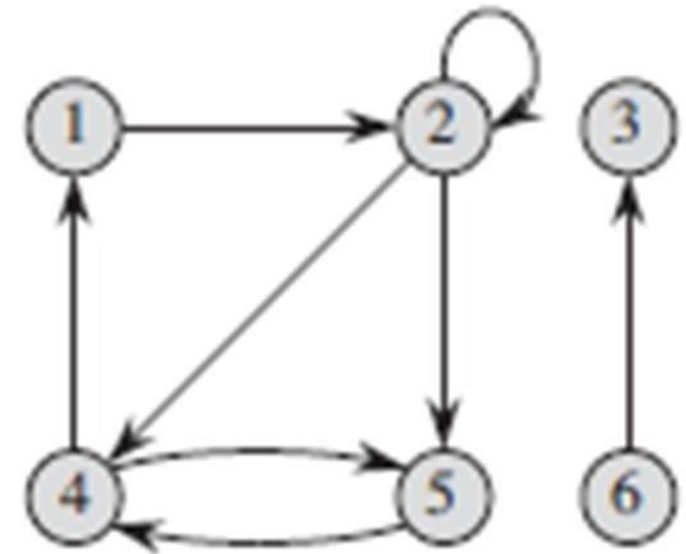
PATH IDENTIFICATION IN UNDIRECTED GRAPHS

- Either BFS or DFS can be used to identify if a path exists between two vertices
- Take the first vertex as a source in BFS (or DFS), follow the standard BFS (or DFS).
- If the second vertex is found in our traversal, then return true else return false.



Introduction –Strongly connected components

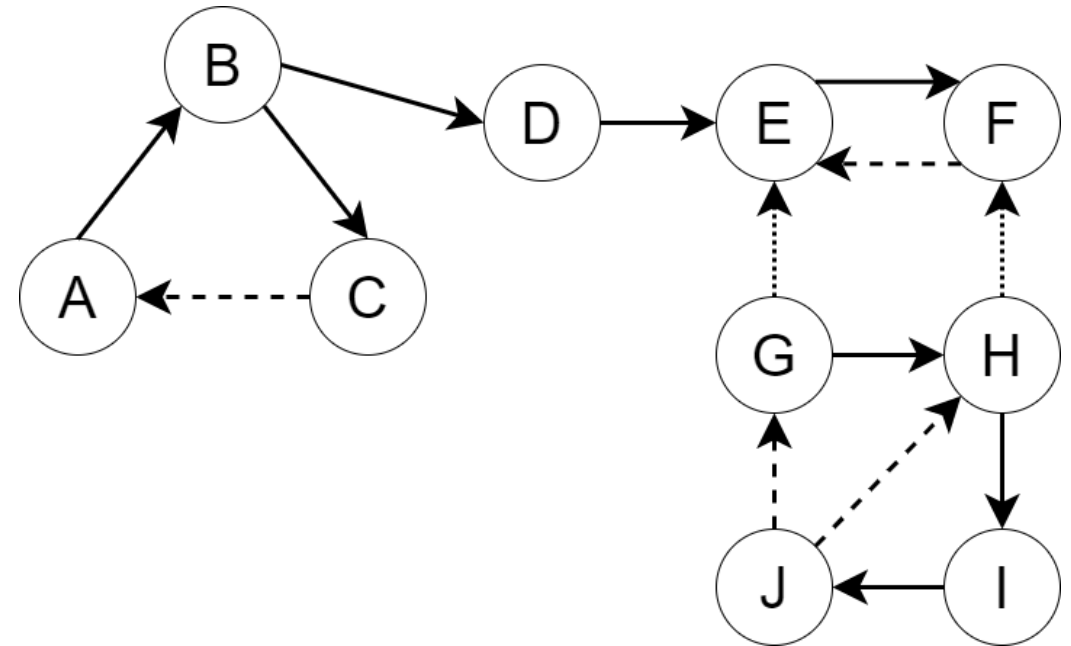
- Any two vertices in **SCC** are mutually reachable, i.e., for any two nodes u, v in the same component, there's a **directed path from u to v and from v to u**
- SCC-(**every vertex is reachable from every other vertex**),
- A vertex is strongly connected with itself





Edges of graph in DFS traversal

- the edges of the graph depending on how DFS discovers them. If DFS processes vertex v and it discovers:
- An unvisited neighbour, u , then edge (v, u) is a tree edge.
- A visited but not yet fully processed neighbor, w , then edge (v, w) is a back edge.
- A fully processed neighbour, t , then edge (v, t) is a cross edge.



DFS visits vertices A, B, C, D, E, F and exits. Then, let's run DFS for G. Now, DFS visits the remaining vertices: G, H, I, and J. The tree edges are depicted with solid lines, the back edges with dashed lines, and the cross edges with dotted lines:



Tarjan's algorithm

- Tarjan's algorithm works by **running DFS** on the graph and outputs each **strongly connected component** when that component's lowest DFS numbered vertex is finished by DFS.
- To help in **outputting the SCCs**, Tarjan's algorithm maintains a stack S of vertices.



Introducing terms dfsNum and dfs LOW link value

- Introduce the concept of a DFS number and DFS low link value BEFORE ENTERING Tarjan's.
- **dfsNum[v] (num(v)/ id):**
 - As DFS traverses a graph it visits the vertices in some order.
 - Starting with the number 0, we **number each vertex according to the order in which DFS visits it.**
 - This is called the DFS number of the vertex, which we can write as dfsNum[v]
- **dfsLow[v](lowest (v) /LOW LINKVALUE):**
 - The **lowest DFS number of any vertex reachable from v (including v).**
 - Low value of a node tells the topmost reachable ancestor



TARJAN'S ALGORITHM

Step 1: Select an unvisited vertex, v . If there're no unvisited vertices, the algorithm terminates.

Step 2: Run DFS for v

Step 3: Go to step 1

- Inside DFS:
- v is marked as visited
- $\text{num}[v]$ (DFSNum) is assigned in the **order of traversal** in DFS
- $\text{lowest}[v]$ (DFS low / LOWLINK VALUE) is initially equal to $\text{num}[v]$
- Next, we go over the v neighbors.
 - If we see an **unvisited neighbor**, u , we invoke DFS for it and, upon returning, update $\text{lowest}[v]$ with $\text{lowest}[u]$ if $\text{lowest}[v] > \text{lowest}[u]$
 - **$\text{lowest}[v] = \min(\text{lowest}[v], \text{lowest}[u]);$**
 - If we see a **visited but not fully processed neighbor**, w , then we have a back edge. In this case, we update $\text{lowest}[v]$ with $\text{num}[w]$ if $\text{lowest}[v] > \text{num}[w]$
 - **$\text{lowest}[v] = \min(\text{lowest}[v], \text{num}[w]);$**
- After we process v 's neighbours, we mark v as processed
- After **v is processed, we check if $\text{num}[v] = \text{lowest}[v]$** . If that's the case, v is the starting vertex of its SCC component. We unwind the stack until we retrieve v . The **unwound vertices belong to the v 's SCC** to find path between two vertices.

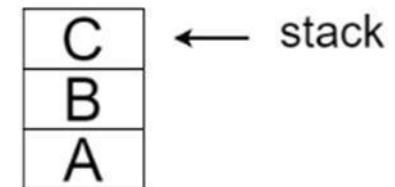
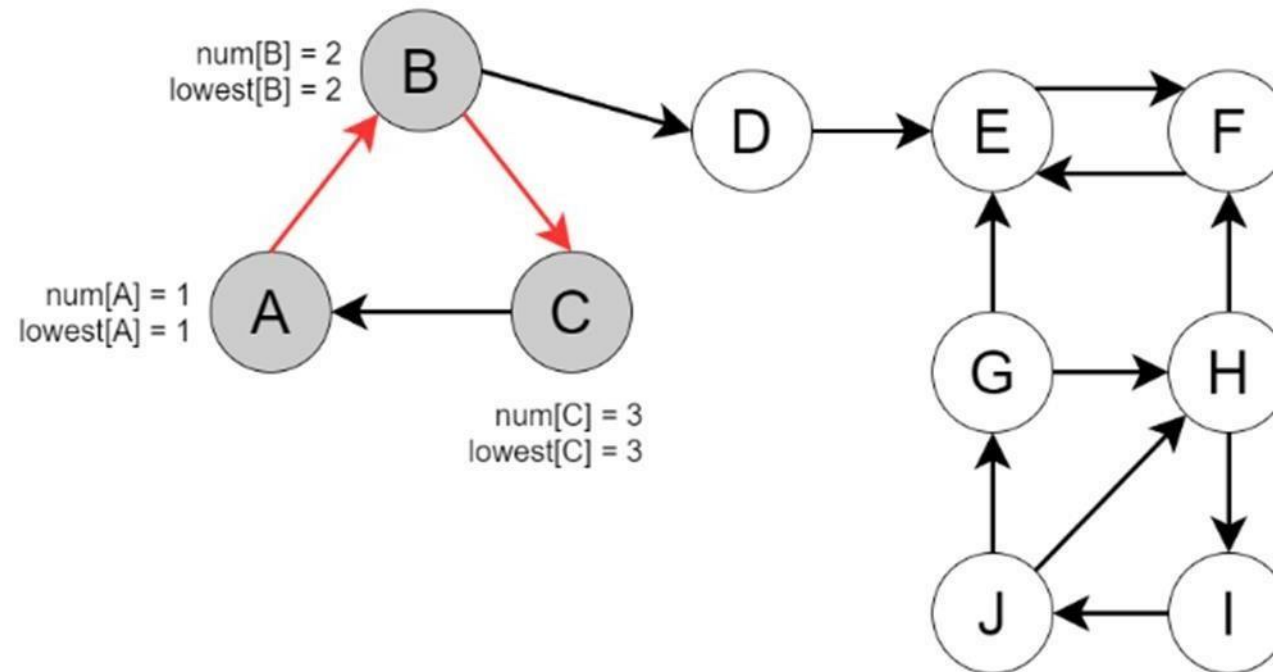


TARJAN'S ALGORITHM

start by running DFS for A.

DFS has visited A, B, and C, but hasn't yet processed back edge (C, A):

$\text{lowest}(v) = \text{num}(v)$





TARJAN'S ALGORITHM

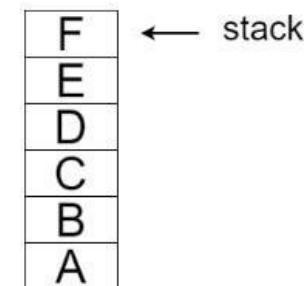
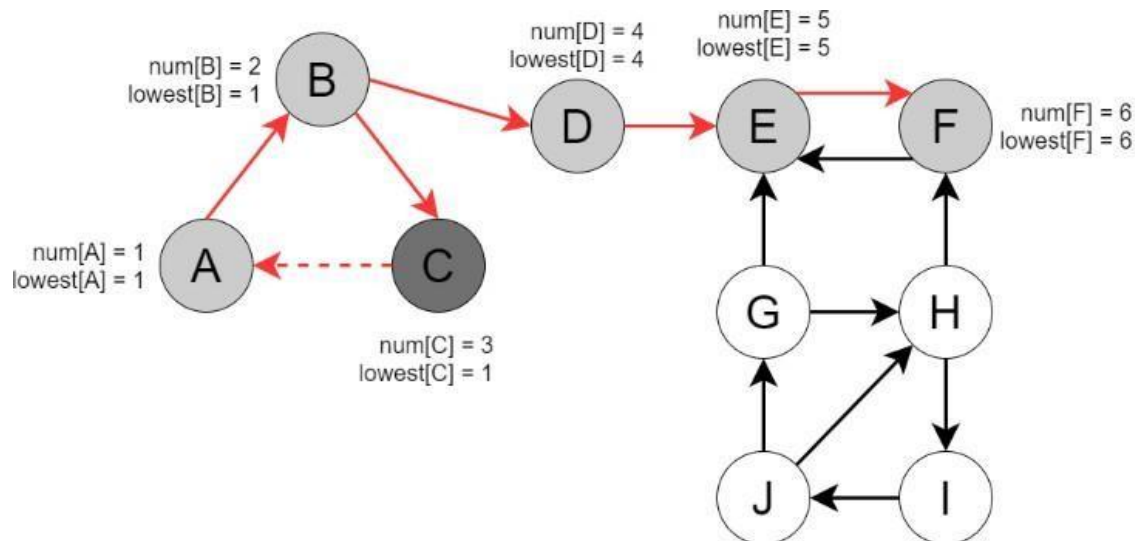
- C has visited neighbour A but not fully processed(backedge)
- SO update $\text{lowest}(C) = \min(\text{lowest}(C), \text{num}(A))$
- Fig shows the state when DFS has processed back edge (C, A), updated $\text{lowest}[C]$, backtracked, and updated $\text{lowest}[B]$. Next, DFS visited the remaining vertices reachable from B:

For node A

$\text{Lowest}(A) = \text{Num}(A)$

For Node B

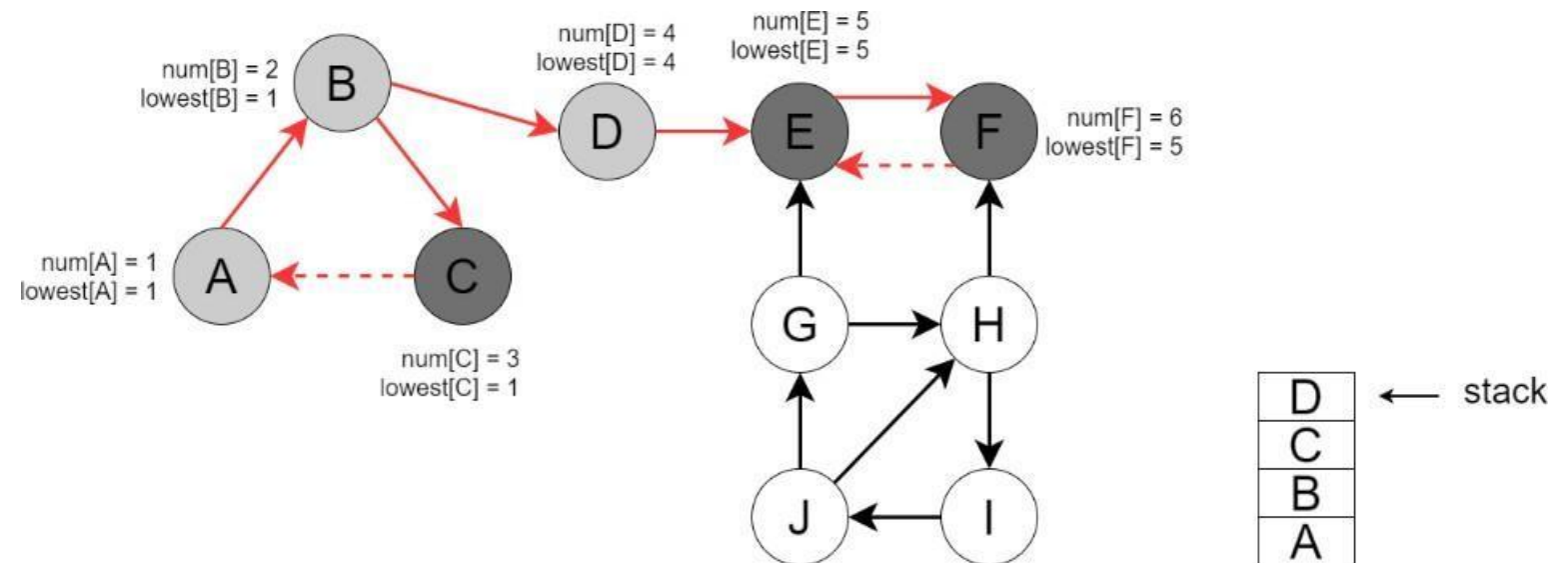
$\text{Lowest}(B) = \min(\text{Lowest}(B), \text{num}(A))$





TARJAN'S ALGORITHM

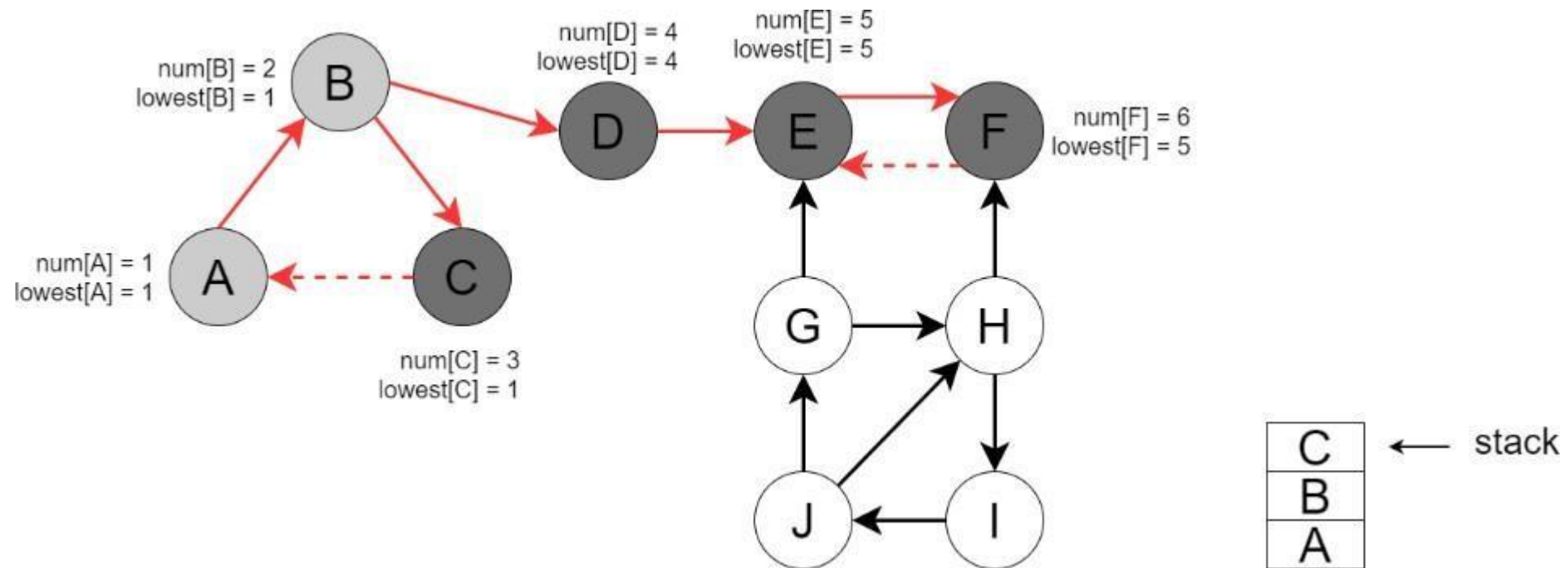
- F has a neighbour that is visited (E) but not fully processed, Back EDGE. HENCE UPDATE $\text{LOWEST}(F) = \min(\text{lowest}(F), \text{num}(F))$
- Then, DFS backtracks from F, and in E $\text{num}(E) = \text{lowest}(E)$
- From Vertex F, pop the vertices if dfsnumber and low link values are same
- (DFS finds the first $\text{SCC} = \{E, F\}$):





TARJAN'S ALGORITHM

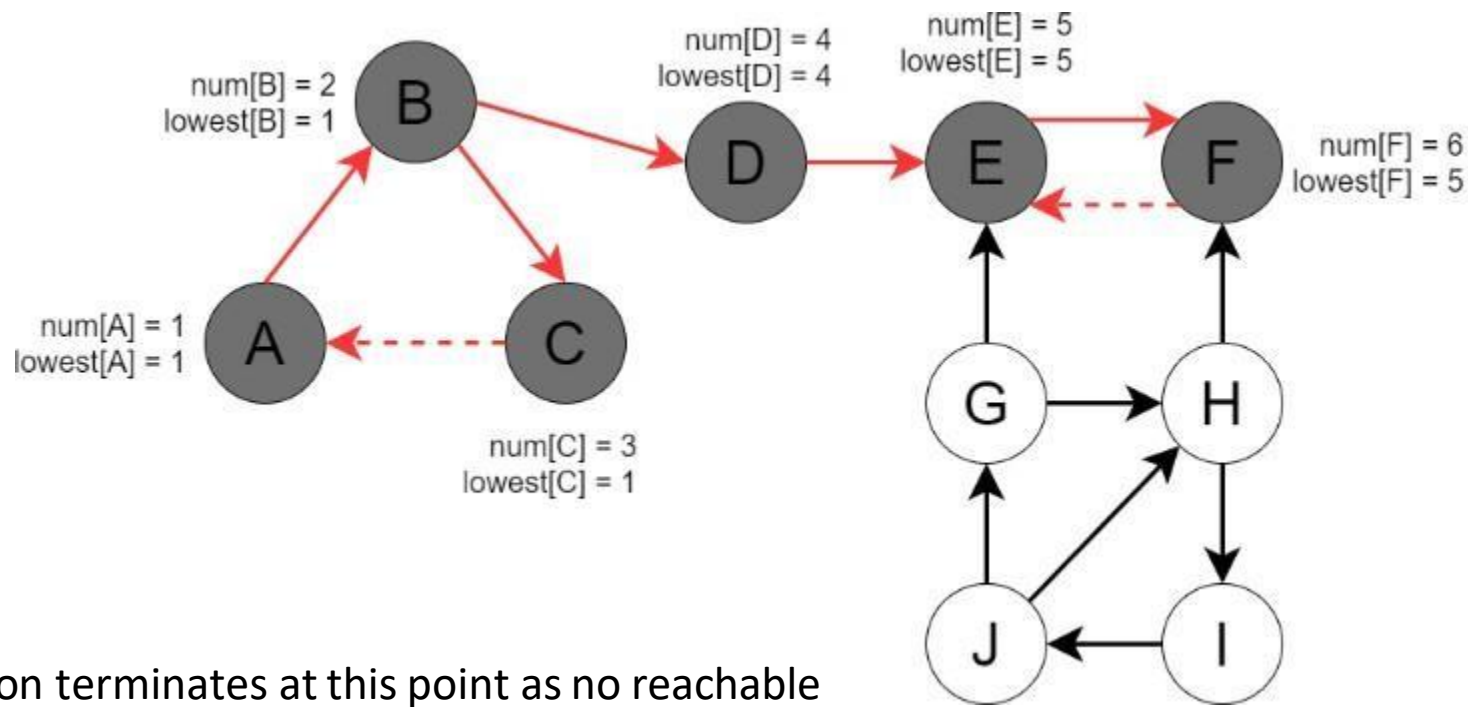
- Then, DFS backtracks to D, and the second **SCC = {D}** is found. (**num(D)=lowest(D)**) (**Condition for scc met**)
- Pop D FROM STACK





TARJAN'S ALGORITHM

- DFS backtracks to B, then to A, and in A DFS finds the third SCC = {A, B, C}
- POP C,B,A



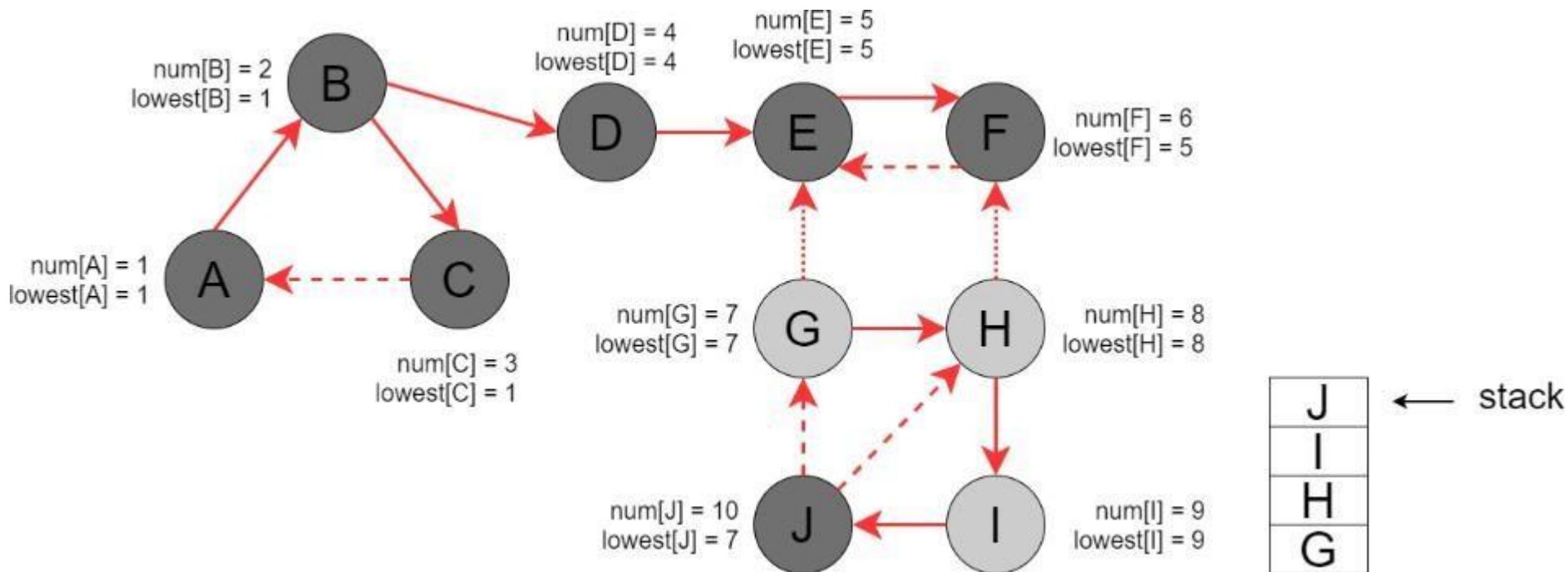
The DFS invocation terminates at this point as no reachable vertices are left.



TARJAN'S ALGORITHM

- Next, we run DFS for an unvisited vertex, G. DFS visits G, H, I, and J, processes back edge (J, G), and updates $\text{lowest}[J]$:

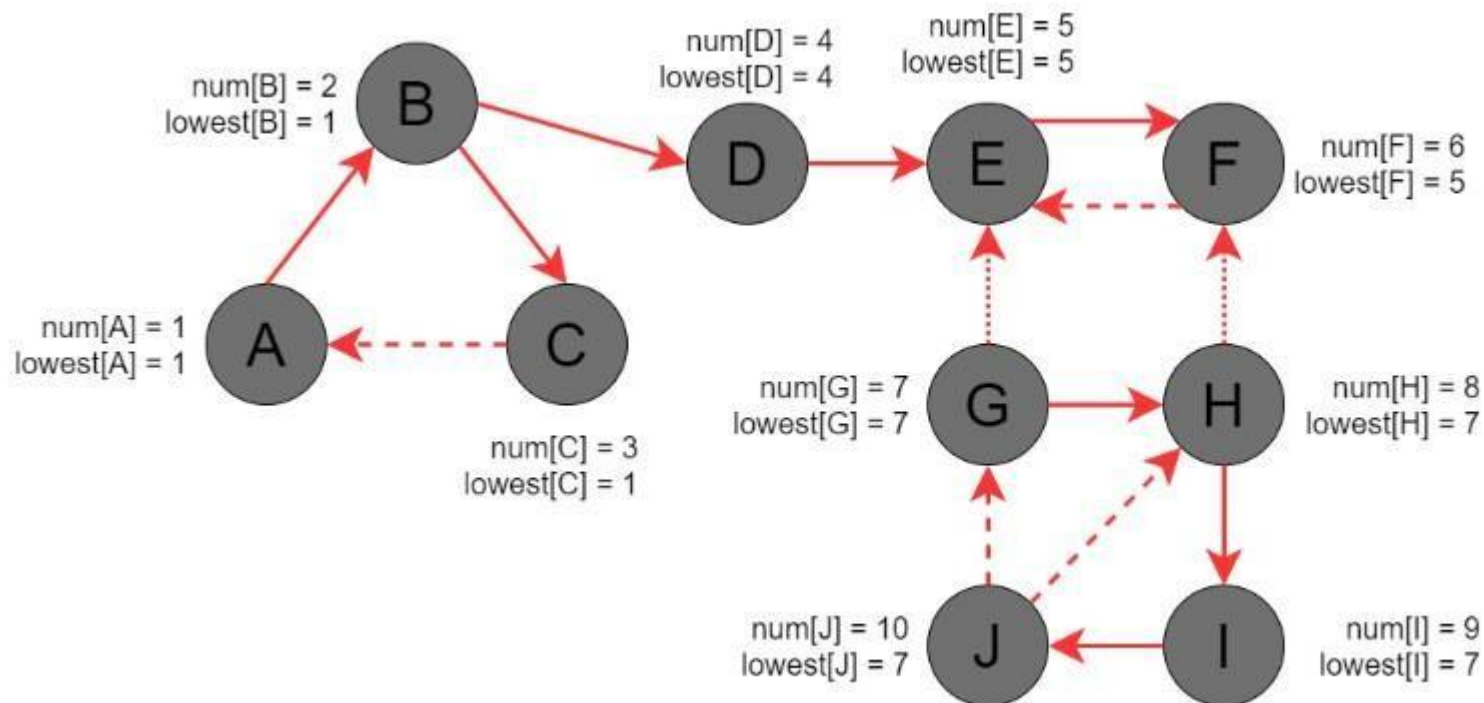
$$\text{Lowest}(J) = \min(\text{Lowest}(J), \text{num}(G))$$





TARJAN'S ALGORITHM (UPDATE LOW LINK VALUE)

- DFS backtracks to G and updates all the vertices on the way:
- $\text{Lowest}(I) = \min(\text{Lowest}(I), \text{num}(G))$
- $\text{Lowest}(H) = \min(\text{Lowest}(H), \text{num}(G))$
- $\text{Lowest}(G) = \min(\text{Lowest}(G), \text{num}(G))$



Finally, when processing G, DFS finds the last $\text{SCC} = \{G, H, I, J\}$ $\text{Lowest}(G) = \text{Num}(G)$



TARJAN'S ALGORITHM

- i – a counter used to assign sequential numbers to the vertices
- $num[]$ – an array of integers holding the vertex numbers, $num[v]$ is the number assigned to v
- $lowest[]$ – an array of integers holding the minimum reachable vertex numbers, $lowest[v]$ is the minimum number of a vertex reachable from v
- $visited[]$ – an array of booleans indicating which vertices have been visited by DFS so far. If $visited[v]$ is TRUE, then DFS has already seen v , but it hasn't necessarily finished processing v
- $processed[]$ – an array of booleans indicating which vertices have been already processed by DFS. If $processed[v]$ is TRUE, then DFS has already finished with v
- s – a stack of vertices used to keep the working set of vertices. s holds all the vertices reachable from the starting vertex. When the algorithm finds an SCC, it will unwind the stack until it gets all the vertices of that SCC



TARJAN'S ALGORITHM

Algorithm 1: DFS for Tarjan's Algorithm

Data: Graph G , current vertex v

Result: Vertices reachable from v are processed, their SCCs are reported

Function DFS(G, v):

```

    num[v] ← i;
    lowest[v] ← num[v];
    i ← i + 1;
    visited[v] ← TRUE;
    s.push(v);
    for  $u \in G.\text{neighbours}[v]$  do
        if visited[u] = FALSE then
            DFS( $G, u$ );
            lowest[v] ← min(lowest[v], lowest[u]);
        else
            if processed[u] = FALSE then
                lowest[v] ← min(lowest[v], num[u]);
            end
        end
    end
    end
    processed[v] ← TRUE;
    if lowest[v] = num[v] then
        scc ← ∅;
        sccVertex ← s.pop();
        while sccVertex ≠ v do
            scc.add(sccVertex);
            sccVertex ← s.pop();
        end
        scc.add(sccVertex);
        Process the found scc in a desired way;
    end
end

```

Algorithm 2: Tarjan's Algorithm

Data: Graph G

Result: SCCs of G are found

Function TARJAN(G):

```

    for  $v \in G.V$  do
        if visited[v] = FALSE then
            DFS( $G, v$ );
        end
    end
end

```

Tarjan's algorithm is a modification of the DFS traversal.

So, the complexity of the algorithm is linear: $O(N + M)$, where N is the number of vertices and M is the number of edges.

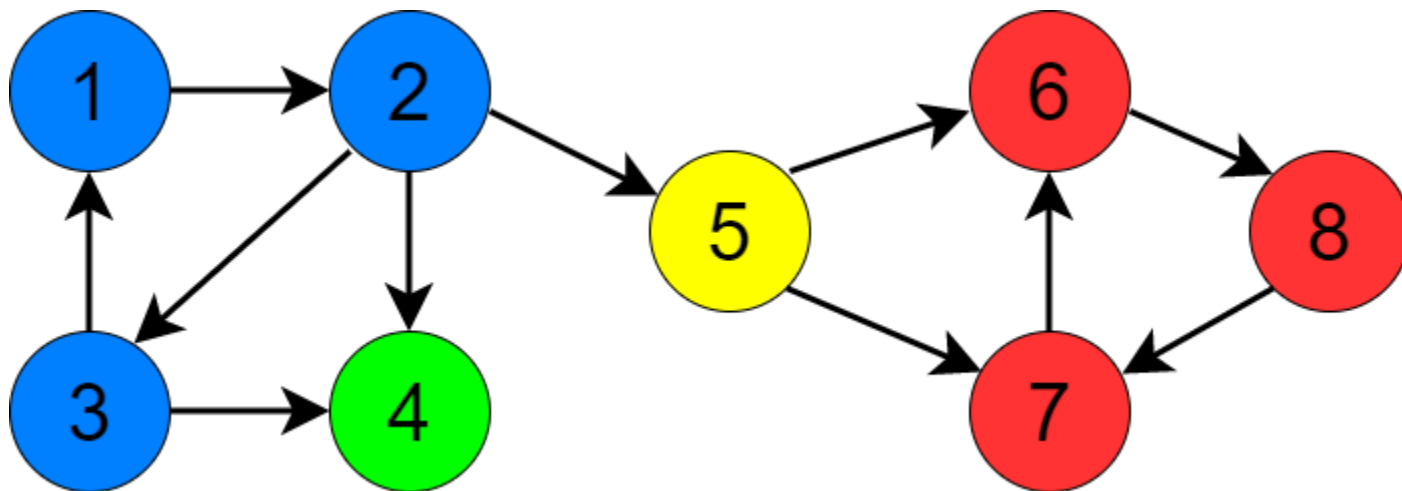
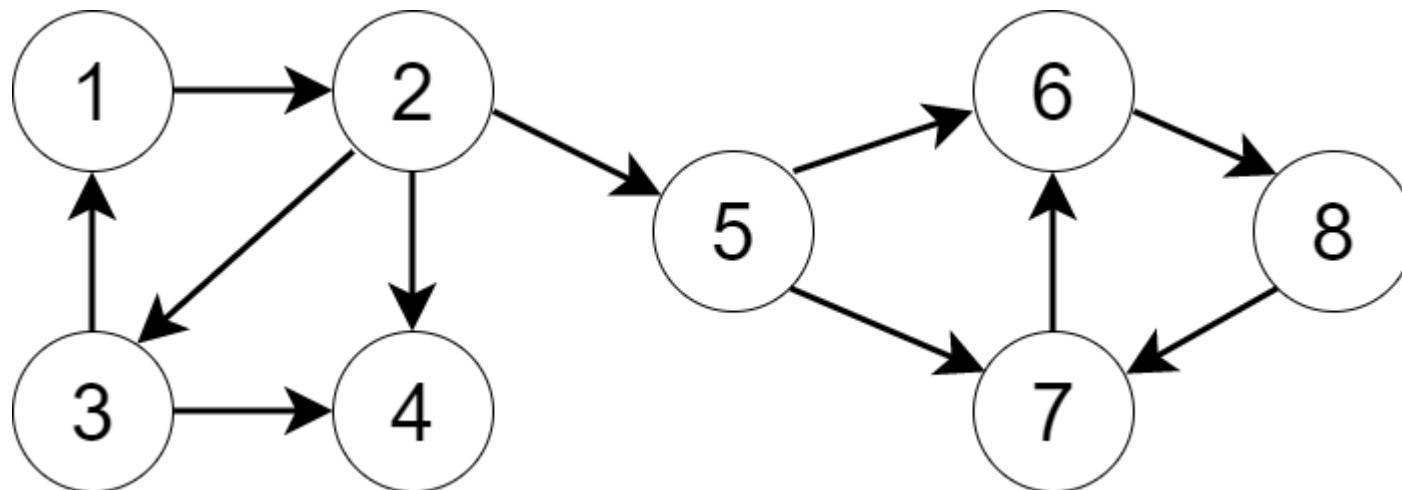


Kosaraju algorithm to find strongly connected component

- Use DFS TWO TIMES
- In the first DFS push vertices in the post order traversal
- Reverse all edges in the graph
- Perform second DFS on vertices in stack
- Each tree in the resulting DFS spanning forest forms SCC



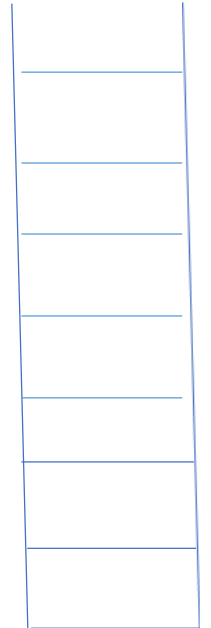
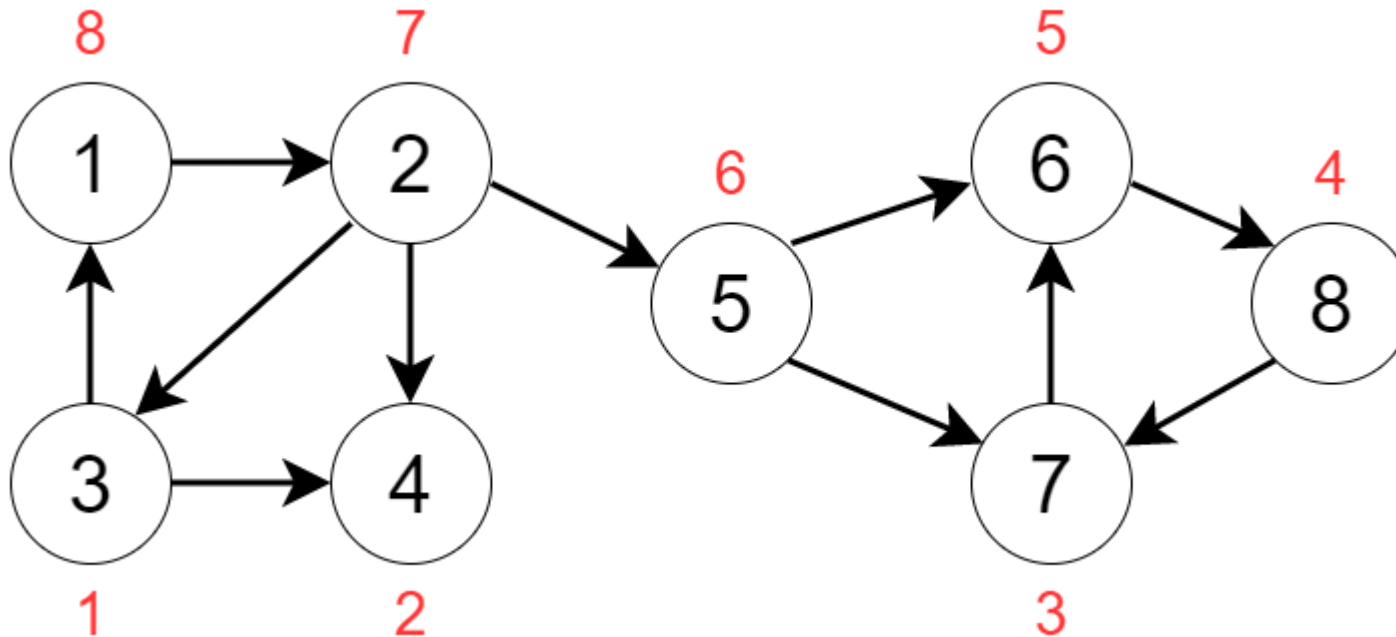
Kosaraju algorithm to find strongly connected component





Kosaraju algorithm to find strongly connected component

- In the first step, we perform a DFS traversal to define the priorities of the vertices.
- DFS Tree- 1 2 5 6 8 7 4 3

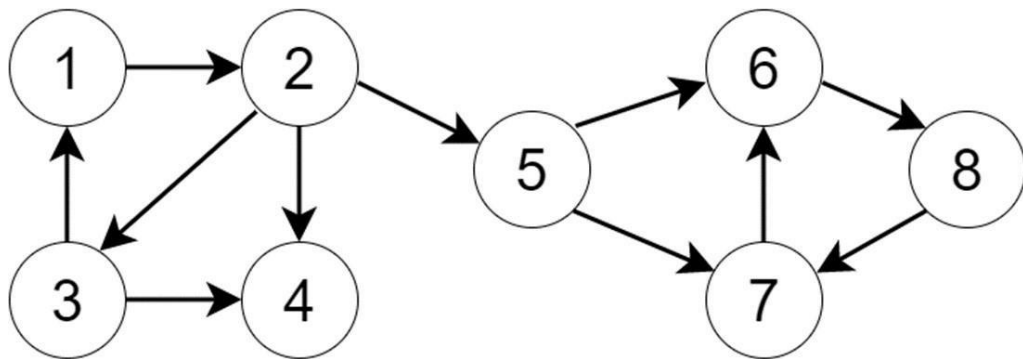


- We **push the vertices onto a stack during DFS traversal according to the priorities**. The stack's content at the end of the process will be: [3, 4, 7, 8, 6, 5, 2, 1].

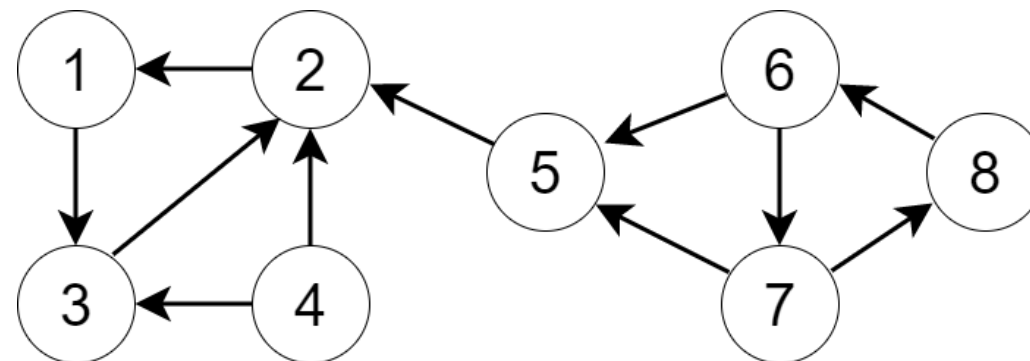


Kosaraju algorithm to find strongly connected component

- Step 2. **Build the transpose graph of G**. The transpose graph is a graph that consists of the same vertices as the original graph, but the **edge directions are reverted**. Here's the transpose graph TG of G



Graph G



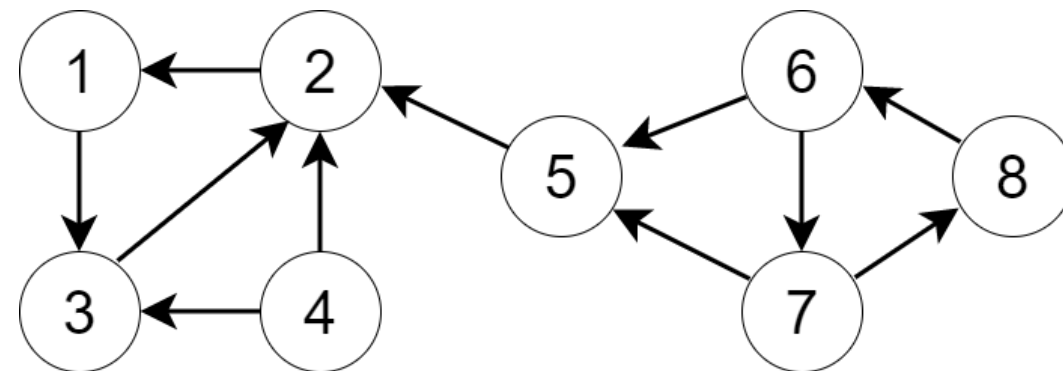
Transposed graph TG
(Inverted edges)



Kosaraju algorithm to find strongly connected component

- In the third step, we **perform a DFS traversal on TG**. Each DFS invocation finds one SCC.
- So, the number of SCCs is equal to the number of times we invoke DFS on TG.
- Each time we call a DFS procedure, we use the non-visited node with the highest priority.

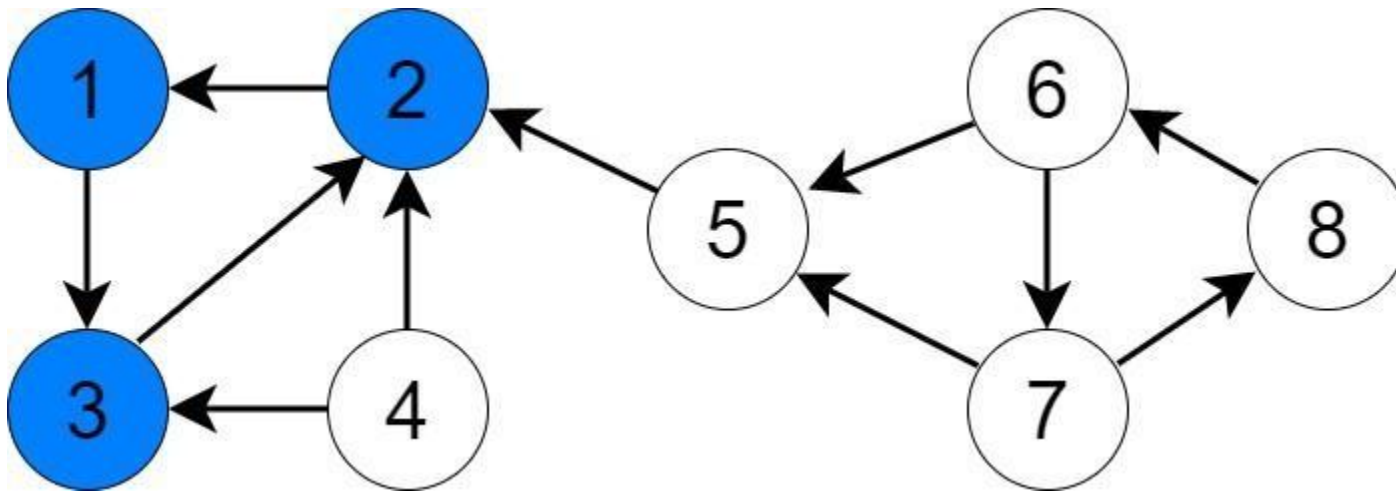
- After step 2, the highest-priority node





Kosaraju algorithm to find strongly connected component

- After step 2, the highest-priority node is **vertex 1**, so we start the first **DFS from that node**. DFS visits vertices **1, 3, and 2** in this order and stops, finding us **one SCC**:

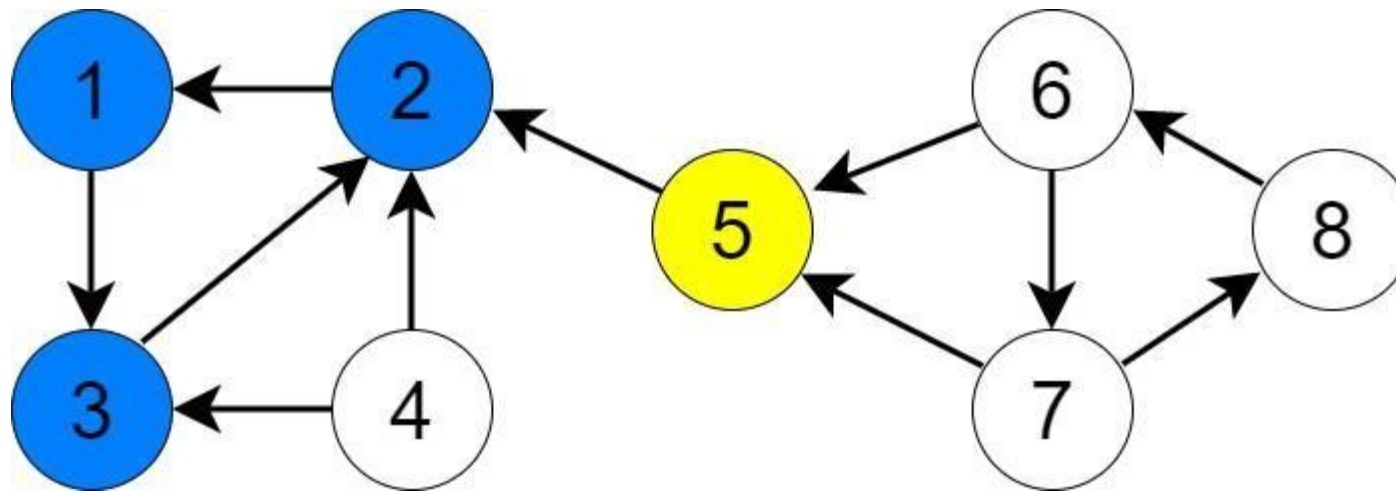


1
2
5
6
8
7
4
3



Kosaraju algorithm to find strongly connected component

- The next vertex with the highest priority is **5**, as vertex 2 is already visited. We call DFS with node 5 as its argument and find the second **SCC containing only that node**:

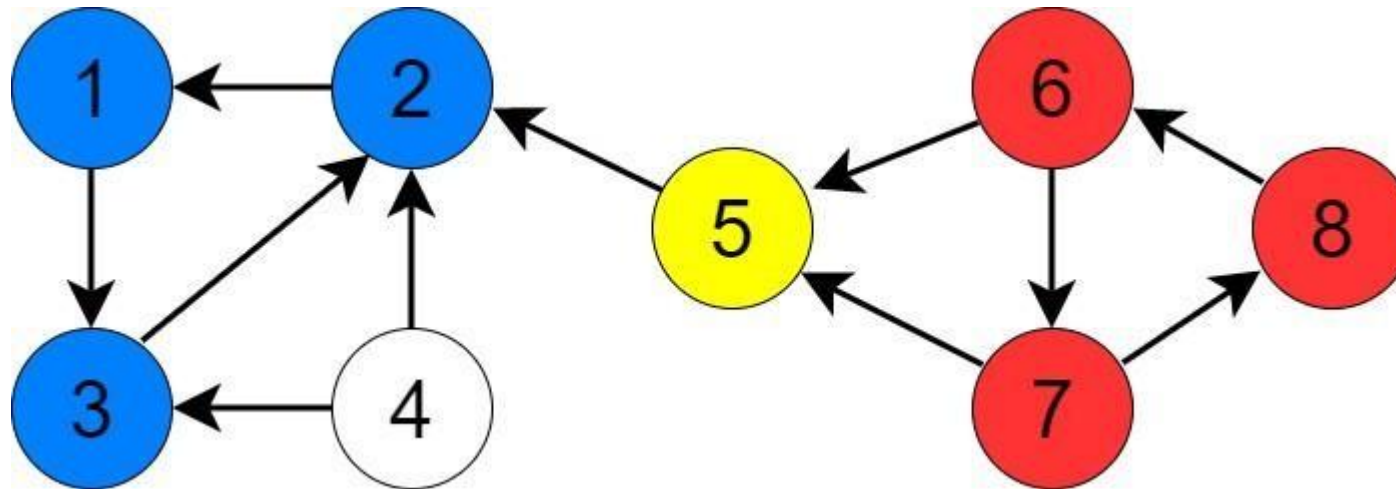


5
6
8
7
4
3



Kosaraju algorithm to find strongly connected component

- Vertex **6** is the next vertex to visit. DFS visits **6, 7, and 8** in this order and returns:

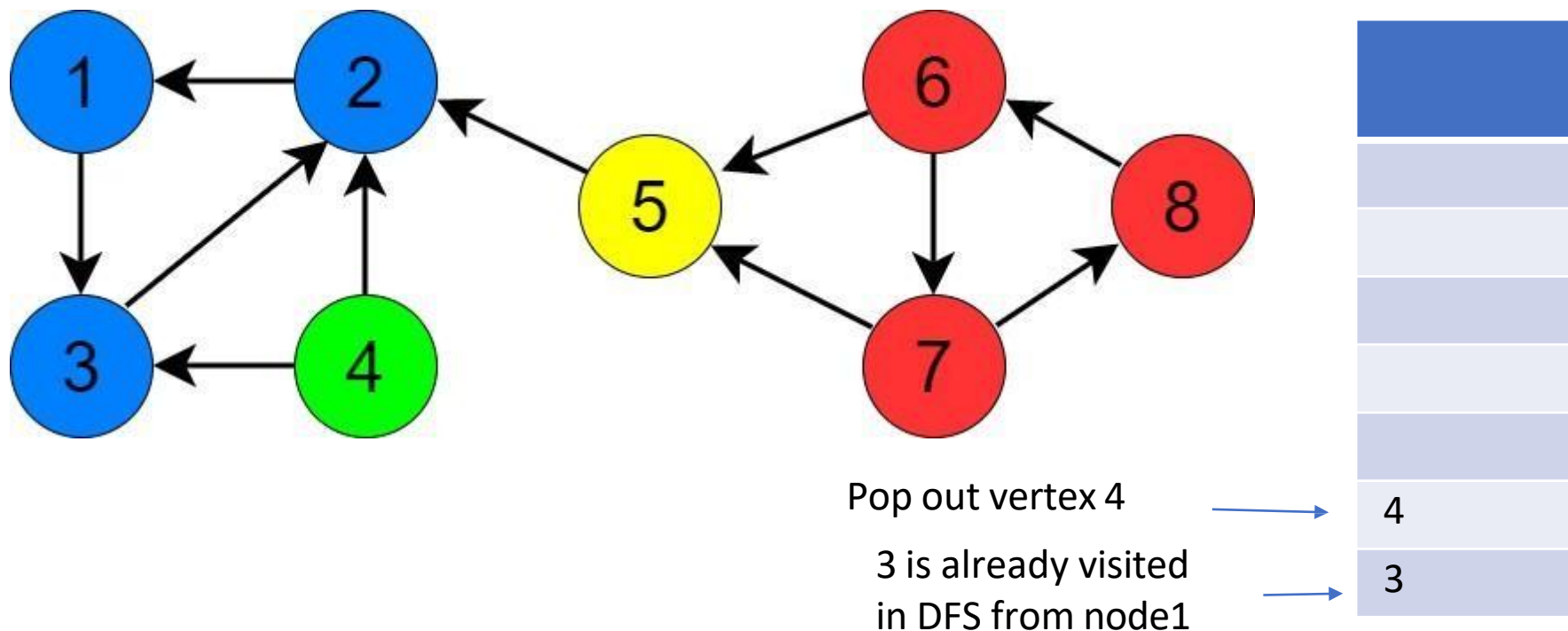


6
8
7
4
3



Kosaraju algorithm to find strongly connected component

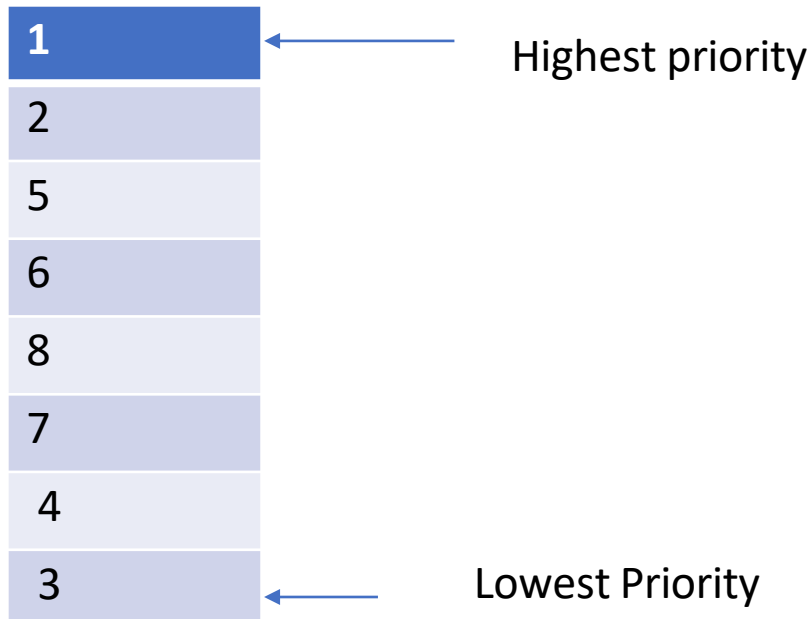
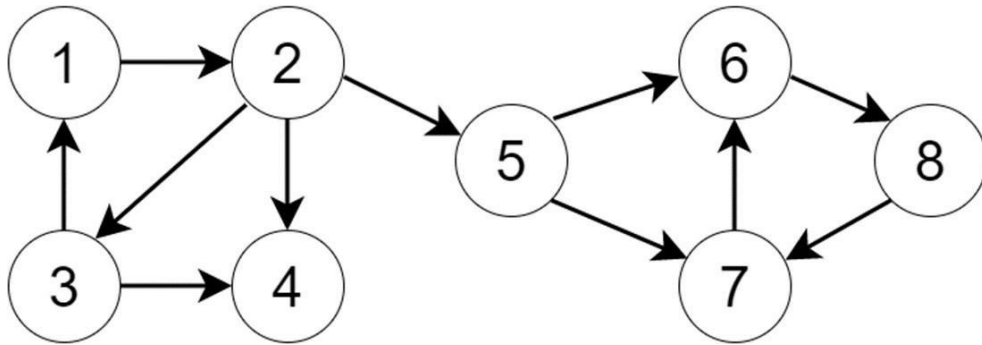
- Lastly, we invoke **DFS for vertex 4** and find the last **SCC containing only that vertex**:





Kosaraju algorithm to find strongly connected component

DFS Tree- 1 2 5 6 8 7 4 3



Algorithm 1: Common DFS

Data: Graph G , vertex v , and array $visited$

Result: Visits all vertices reachable from v

Function DFS($G, v, visited$):

```
     $visited[v] \leftarrow \text{TRUE};$   
    process vertex  $v$ ;  
    for  $u \in G.neighbours[v]$  do  
        if  $visited[u] = \text{FALSE}$  then  
            DFS( $G, u, SCC, visited$ );  
        end  
    end
```

Algorithm 2: DFS With Vertex Priorities

Data: Graph G , vertex v , stack $priority$ to push vertices by their priorities, the first pushed vertex has the least priority, and array $visited$

Result: Visits all vertices reachable from v and puts them into $priority$ stack according to the visited time

Function PDFS($G, v, priority, visited$):

```
     $visited[v] \leftarrow \text{TRUE};$   
    for  $u \in G.neighbours[v]$  do  
        if  $visited[u] = \text{FALSE}$  then  
            PDFS( $G, u, priority, visited$ )  
        end  
    end  
     $priority.push(v);$ 
```



Kosaraju algorithm to find strongly connected component

Algorithm 3: Transform Graph Creation

Data: Graph G

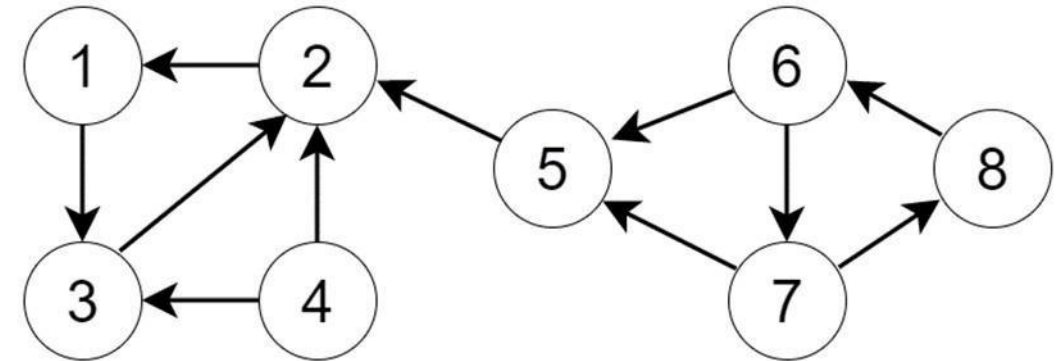
Result: Graph TG which is the transform graph of G

Function TRANSFORM(G):

```

     $TG \leftarrow$  new graph of  $N$  vertices;
    for  $v \in G.V$  do
        for  $u \in G.neighbours[v]$  do
             $TG.neighbours[u].add(v)$ ;
        end
    end
    return  $TG$ ;

```



1

2

5

6

8

7

4

3

Algorithm 4: Kosaraju's Algorithm

Data: Graph G

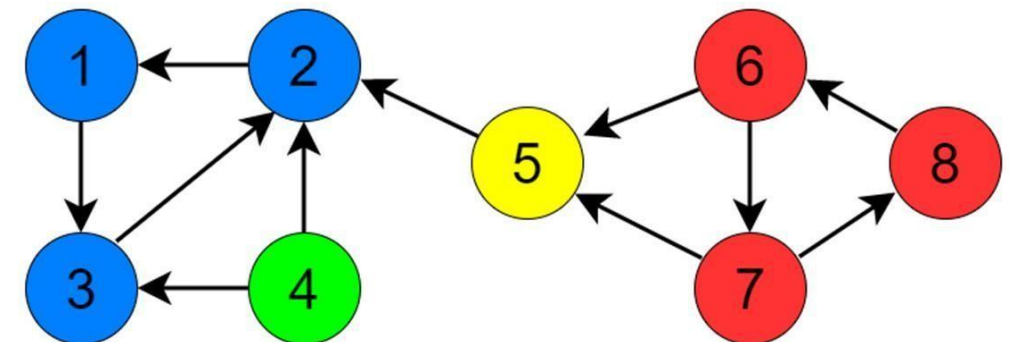
Result: Strongly connected components are found

Function KOSARAJU(G):

```

    visited  $\leftarrow$  boolean array of size  $N$  containing FALSE values;
    priority  $\leftarrow$  vertex stack;
    for  $v \in G.V$  do
        if visited[ $v$ ] = FALSE then
            PDFS( $G, v, priority, visited$ );
        end
    end
     $TG \leftarrow$  TRANSFORM( $G$ );
    clear array visited;
    while priority stack is not empty do
         $v \leftarrow$  priority.pop();
        if visited[ $v$ ] = FALSE then
            DFS( $TG, v, visited$ );
            process current SCC;
        end
    end

```





Complexity-Kosaraju algorithm

Step 1 is a DFS traversal of G , which runs in $O(N + M)$

Step2 For transposing G , we need to go over all the vertices and edges, so it's another $O(N + M)$ operation

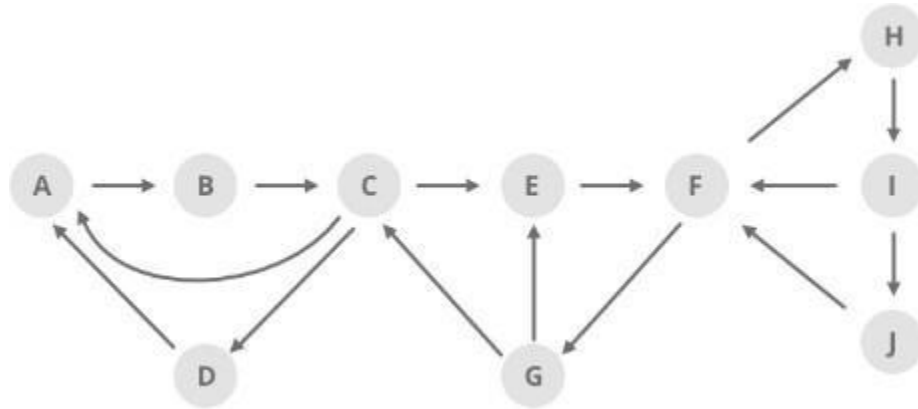
Step 3 is a DFS traversal of TG , which also runs in $O(N + M)$

Thus, the overall complexity of Kosaraju's algorithm is $O(N + M)$

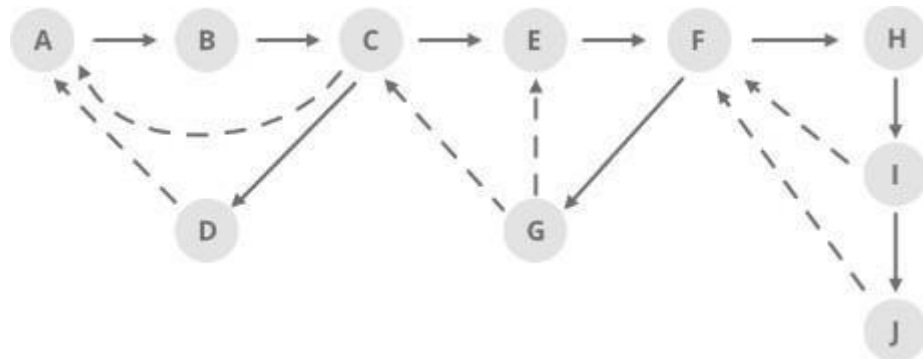


Practice problem

Find the SCC's in the given graph .Apply Tarjan's algorithm



If there are multiple back edges in the subtree that take us to different ancestors, then we take the one with the minimum Disc value (i.e. the topmost one). If we look at node F, it has two subtrees. Subtree with node G takes us to E and C. The other subtree takes us back to F only. Here topmost ancestor is C where F can reach and so the Low value of F is 3 (The Disc value of C).



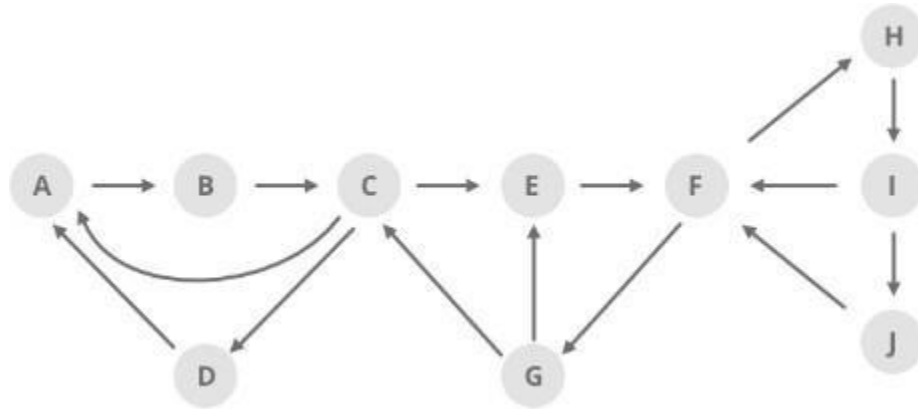
DFSnum
DFSLOW

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10
1	1	1	1	3	3	3	6	6	6



Practice problem

Find the SCC's in the given graph .Apply Tarjan's algorithm



DFSnum

DFSLOW

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10
1	1	1	1	3	3	3	6	6	6

